

319292709

Grupo: 03



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA
INGENIERÍA EN COMPUTACIÓN



TEMAS SELECTOS DE INGENIERIA EN COMPUTACIÓN I

Fundamentos Técnicos VFX

GRUPO 3

SEMESTRE 2026 - 2



MANUAL TÉCNICO



Num de Cuenta: 319292709

FECHA DE ENTREGA: 21 de mayo de 2026

Contenido

Introducción	4
Objetivo.....	4
Alcance del proyecto	5
Alcances funcionales.....	5
Alcances técnicos	5
Alcances narrativos	5
Limitaciones del proyecto.....	5
Limitantes que se presentaron	6
Planeación del proyecto	6
Etapas del desarrollo	6
Diagrama de Gantt.....	8
Costos del proyecto	9
Tecnologías utilizadas	10
Requerimientos técnicos.....	10
Shader personalizado	10
Efectos visuales.....	11
Iluminación técnica	12
Postprocesado	13
Optimización y rendimiento.....	14
Arquitectura del proyecto	15
Modelos.....	15
Personajes.....	15
ESCENARIO	18
RIG	22
ANIMACIÓN	23
EN UNITY.....	24
EVENTOS.....	25
ARBOL DE ANIMACIÓN.....	26

PLAYER INPUT	29
COMBATE.....	30
ESPADA	30
MAGIA	31
ENEMIGOS	31
INTERFAZ	32
cámaras.....	34
VOCES.....	34
Música.....	37
VIDEO_INTRO.....	37
ESCENAS	38
SCRIPTS.....	39
MINIJUEGOS	41
SHADERS.....	41
Río	41
Fuego.....	42
SISTEMA DE PARTICULAS	42
Luciérnaga.....	42
Fuego	43
CONCLUSIONES	44

Introducción

El proyecto presentado fue desarrollado utilizando el motor de videojuegos Unity, teniendo como objetivo la creación de una escena interactiva que integrara diversas herramientas y técnicas vistas a lo largo del curso, así como otras adicionales investigadas e implementadas durante el desarrollo, tales como efectos visuales (VFX), shaders, sistemas de partículas, animaciones y mecánicas de interacción.

El videojuego fue diseñado bajo una estética low poly, decisión tomada con el propósito de mantener un mejor rendimiento dentro del entorno gráfico, utilizando modelos tridimensionales de menor complejidad geométrica sin perder la ambientación visual deseada. Además, el proyecto plantea un mundo de fantasía donde el jugador puede explorar, utilizar armas cuerpo a cuerpo y ejecutar habilidades mágicas como parte de las mecánicas de combate.

Como escenario principal se implementó un bosque nocturno, buscando generar una ambientación inmersiva mediante el uso de iluminación dinámica, partículas ambientales y shaders aplicados a distintos elementos del entorno.

Asimismo, el proyecto incorpora sistemas de animación para personajes, enemigos y objetos interactivos. Varias de estas animaciones hacen uso de eventos para sincronizar acciones específicas como sonidos, efectos visuales, activación de daño y ejecución de mecánicas dentro del juego, permitiendo una interacción más fluida y dinámica.

De igual manera, se implementaron distintos sistemas funcionales como interfaz gráfica, diálogos con NPCs, misiones, sistema de combate, y elementos interactivos. Finalmente contando con una narración que al tratarse de un DEMO, en el proyecto solo se presenta el inicio de esta historia.

Objetivo

Desarrollar un videojuego estilo RPG en Unity donde el usuario pueda adentrarse en la historia del personaje principal a través de la exploración de un mundo ambientado en fantasía. El jugador podrá enfrentarse a diferentes enemigos utilizando un sistema de combate basado en armas cuerpo a cuerpo y magia.

El videojuego busca ofrecer una experiencia inmersiva mediante mecánicas de exploración, interacción con personajes, cumplimiento de misiones y progresión del personaje, permitiendo que el jugador descubra la historia mientras avanza a lo largo del juego. Por lo que se busca crear el DEMO para este videojuego.

Alcance del proyecto

Alcances funcionales

- Desarrollo de un sistema de movimiento y exploración en tercera persona. Por lo tanto, se logró implementar con éxito la cámara muy similar a los videojuegos en el mercado.
- Implementación de combate mediante armas y ataques mágicos.
- Sistema de vida, daño y curación mediante pociones.
- Interacción con NPCs mediante diálogos.
- Implementación de misiones principales.
- Aparición de enemigos y mecánicas de combate.
- Patrullaje de enemigos.
- Uso de cofres, objetos interactivos y eventos dentro del escenario.
- Integración de interfaz gráfica para mostrar vida, misiones y objetos. Y otra para la pausa, menú principal, etc.

Alcances técnicos

- Desarrollo del videojuego utilizando el motor gráfico Unity.
- Programación de mecánicas mediante scripts en C#.
- Implementación de animaciones para personajes, enemigos y ataques.
- Uso de efectos visuales y partículas para magia y combate.
- Integración de audio, música ambiental y efectos sonoros.
- Implementación de control de escenas
- Uso de prefabs y sistemas reutilizables para optimizar el desarrollo.

Alcances narrativos

- Desarrollo de una historia principal enfocada en un bosque y criaturas hostiles.
- Construcción de narrativa mediante diálogos, notas y exploración.
- Creación de personajes secundarios.
- Ambientación de fantasía oscura con elementos de misterio y supervivencia.

Limitaciones del proyecto

- El videojuego está diseñado únicamente para modo de un solo jugador.
- El proyecto se desarrolla para plataforma PC.
- El sistema de inventario y progresión será básico en esta versión.

- El mapa y número de enemigos estarán limitados a una demostración funcional del juego.

Limitantes que se presentaron

- El equipo de cómputo utilizado presentó limitaciones de rendimiento al trabajar con escenarios amplios, efectos visuales y múltiples objetos dentro de Unity.
- Para el diseño que se planteó y al ser solo una persona, se tuvo que adaptar al tiempo haciendo ciertas dinámicas como el combate más sencillo.
- La optimización del videojuego se limitó a pruebas en PC, por lo que no se comprobó su rendimiento en equipos con diferentes características técnicas ni en otras plataformas.

Planeación del proyecto

Etapas del desarrollo

El desarrollo del videojuego se llevó a cabo mediante distintas etapas de forma secuencial.

1. Planeación e ideación del proyecto

En la primera etapa se definió la idea general del videojuego, estableciendo el género, ambientación y estilo visual que tendría el proyecto. Asimismo, se realizó una lluvia de ideas para definir las mecánicas y la historia.

2. Diseño conceptual y documentación

Posteriormente, se elaboraron distintos elementos de diseño necesarios para estructurar el proyecto, como es la elaboración de la historia con el guion. En base a lo anterior se procedió a el diseño inicial del mapa y la planeación de las mecánicas.

3. Prototipado inicial

En esta etapa se desarrolló un prototipo funcional con las mecánicas básicas del videojuego. Se implementaron sistemas iniciales como:

- Movimiento del personaje.
- Cámara

Esta etapa representó un reto durante el desarrollo del proyecto, ya que inicialmente se siguió un tutorial para la implementación de la cámara. Sin embargo, al avanzar en

el desarrollo se observó que el resultado obtenido no cumplía con las expectativas esperadas.

Por ello, se optó por comprender y configurar manualmente las propiedades de la cámara, incluyendo las herramientas de Cinemachine, especialmente el sistema Cinemachine FreeLook. Esto permitió obtener un movimiento más libre y dinámico de la cámara, así como una mejor interacción y detección con los colliders dentro del escenario, mejorando la experiencia de juego y el control del personaje.

- Sistema de vida.
- Inventario básico.
- Sistema simple de combate.

El objetivo de esta fase fue comprobar la funcionalidad general del juego antes de integrar elementos más complejos.

4. Desarrollo visual y artístico

Una vez establecidas las mecánicas básicas, se comenzó la parte visual del proyecto. Durante esta fase se trabajó en:

- Modelado y adaptación de personajes.
- Diseño de props y elementos del escenario.
- Implementación de materiales y shaders.
- Integración de partículas y efectos visuales.

5. Animación e interacción

En esta etapa se incorporaron animaciones tanto para el personaje principal como para enemigos y objetos interactivos. Esto se creó mediante un árbol de animaciones que se explicará a más a detalle posteriormente, además que ciertas animaciones contaron con eventos para sincronizar acciones del juego. Además de las interacciones tanto de los objetos como de los NPC.

6. Implementación de mecánicas avanzadas

Posteriormente, al haber experimentado con mecánicas básicas para el videojuego se pasó a un sistema más complejo como fue el sistema de combate cuerpo a cuerpo, el sistema de magia, los enemigos con comportamiento básico, diálogos interactivos y el sistema de voces y sonidos ambientales. Así como los efectos visuales en las animaciones.

7. Optimización y pruebas

Las pruebas no solo fueron en esta etapa, estas estuvieron presentes en todo momento ya que fue desarrollado en base de prueba y error. Se planteaba una mecánica y se probaba buscando los errores y encontrar el resultado esperado. Y en caso de la optimización Unity tiene herramientas que ayudan en este aspecto.

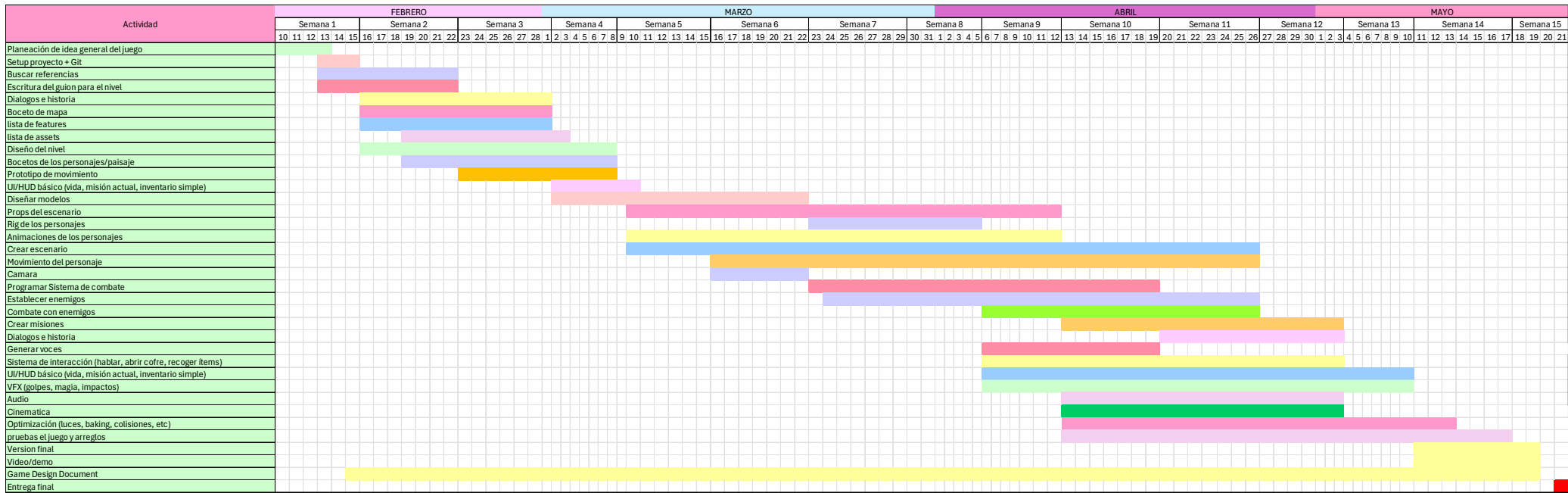
8. Integración final y entrega

Finalmente, se realizaron los últimos ajustes visuales y técnicos del videojuego, integrando elementos como:

- Cinemática introductoria.
- Audio final.
- Versión de demostración.

Diagrama de Gantt

A continuación, se presenta el diagrama de Gantt que resume todo el proceso de desarrollo para este proyecto:



Costos del proyecto

Computadora

$$\frac{40,000(93 \text{ días})}{(5)(365)} = 2038.36$$

Disco duro

$$\frac{1549(93 \text{ días})}{(5)(365)} = 78.9$$

Recurso	Cantidad	Costo	Categoría	Precio
Asus Zenbook UX534FTC	1	\$ 2,038.36	Hardware	\$ 2,038.36
ElevenLab	1	\$ 192.00	Software	\$ 192.00
Blender	1	\$ -	Software	\$ -
Unity	1	\$ -	Software	\$ -
Visual Studio Code	1	\$ -	Software	\$ -
Github	1	\$ -	Software	\$ -
Disco duro externo SSD	1	\$ 78.90	Hardware	\$ 78.90
Control PS5	1		Hardware	\$ -
meshia	1 mes	\$180.00	Software	\$ 180.00
Desarrollador C#, Unity, etc	4	\$25,000.00	Recursos Humanos	\$ 100,000.00
Tester	4	\$25,000.00	Recursos Humanos	\$ 100,000.00
Convertio.co	1	\$ -	Software	\$ -
CapCut	1	\$ -	Software	\$ -
Imovie	1	\$ -	Software	\$ -
Chatgpt	4	\$ 420.00	Software	\$ 1,680.00
				\$
total				204,169.26

Tecnologías utilizadas

- **Motor gráfico**

Unity

- **Lenguaje de programación**

C#

- **Modelado 3d**

Blender

- **Generador de modelos**

Meshy.ia

- **Edición de Video**

Capcut y iMovie

- **Convertidor de formatos**

Convertio.co

- **Genrador de voces y video**

ElevenLabs

- **Generador de imágenes**

ChatGPT y Copilot

- **Control de verisionamiento**

Github

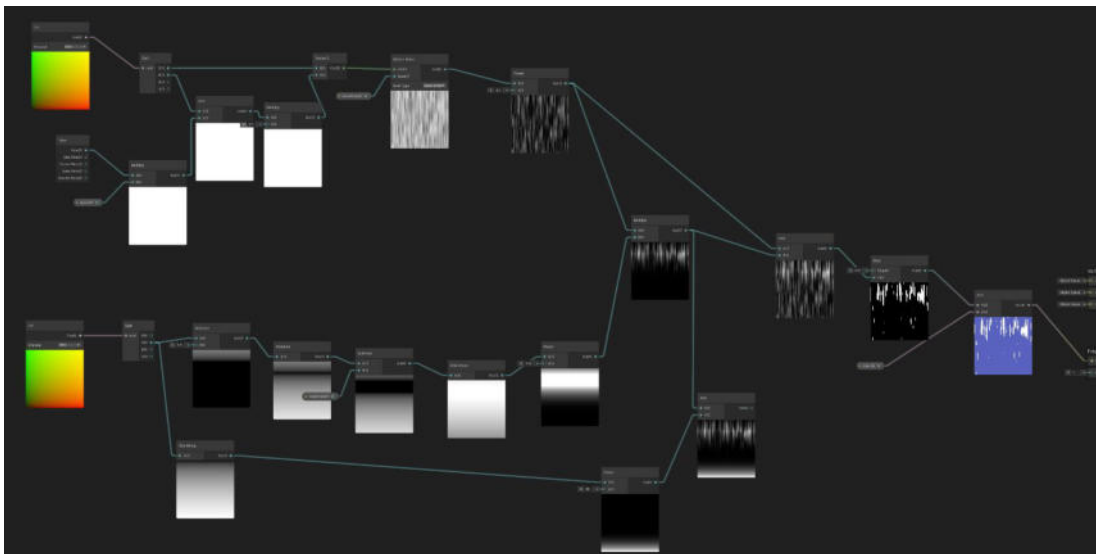
Requerimientos técnicos

A continuación, se mencionarán los requisitos técnicos del proyecto.

Shader personalizado

Para el shader creado hice seis versiones de agua, ya que tuve el problema de que el plano que era para el río sus UV o su forma no daba lo que se veía previamente en el shaderGraph. Por lo que hice varias técnicas hasta que la siguiente fue la que me agradó:

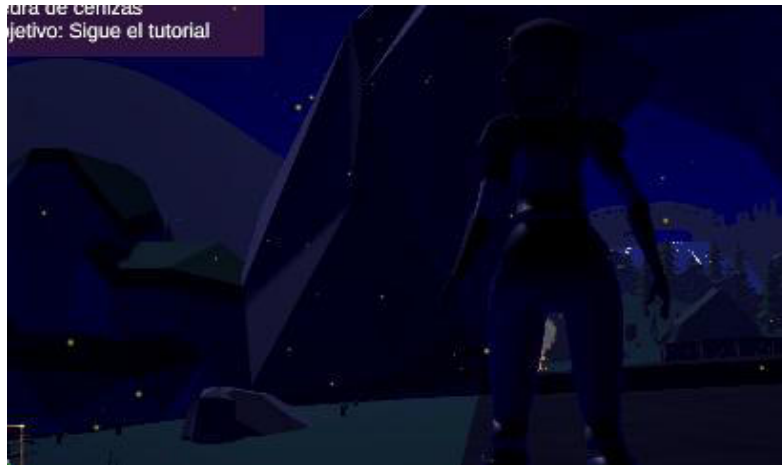
Este shader de río fue creado utilizando varias técnicas para simular agua en movimiento de estilo stylized o low poly. Primero, las coordenadas UV se desplazan usando el nodo Time para que las texturas y el ruido parezcan fluir como una corriente. Después se aplica un Simple Noise procedural que genera variaciones orgánicas similares a turbulencia o espuma, y mediante nodos como Power se aumenta el contraste para hacer más visibles esas formas. En la parte inferior se crea una máscara vertical usando operaciones como Split, Subtract, Absolute y One Minus, lo que permite controlar dónde aparecen los efectos del agua, normalmente concentrándolos en el centro del río. Luego se combinan el noise y las máscaras con nodos Multiply y Add, y finalmente el nodo Step transforma los degradados suaves en cortes bruscos que simulan espuma o reflejos estilizados. El shader también usa Alpha para controlar transparencia y una conexión al Vertex para mover ligeramente la geometría, haciendo que el agua tenga pequeñas olas y no se vea completamente plana.



En cuestión de los demás shaders, fueron descargados o hechos siguiendo tutoriales para poder efectuarlos.

Efectos visuales

Para darle el efecto de magia una ayuda fue el sistema de partículas para simular las luciérnagas.

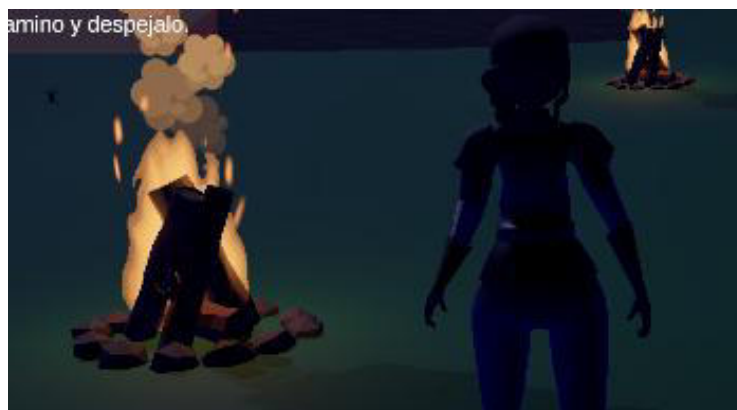


U otros donde se hacían eventos en las animaciones para que cuando sucedieran pudieran aparecer los efectos especiales como es el golpe o curarse. Los eventos son explicados en el apartado de la animación.



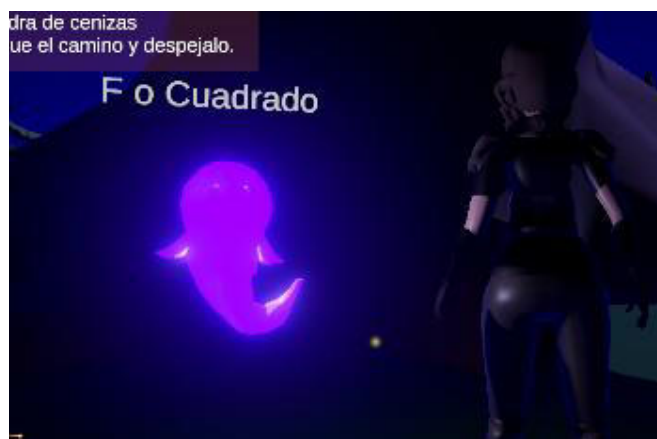
Iluminación técnica

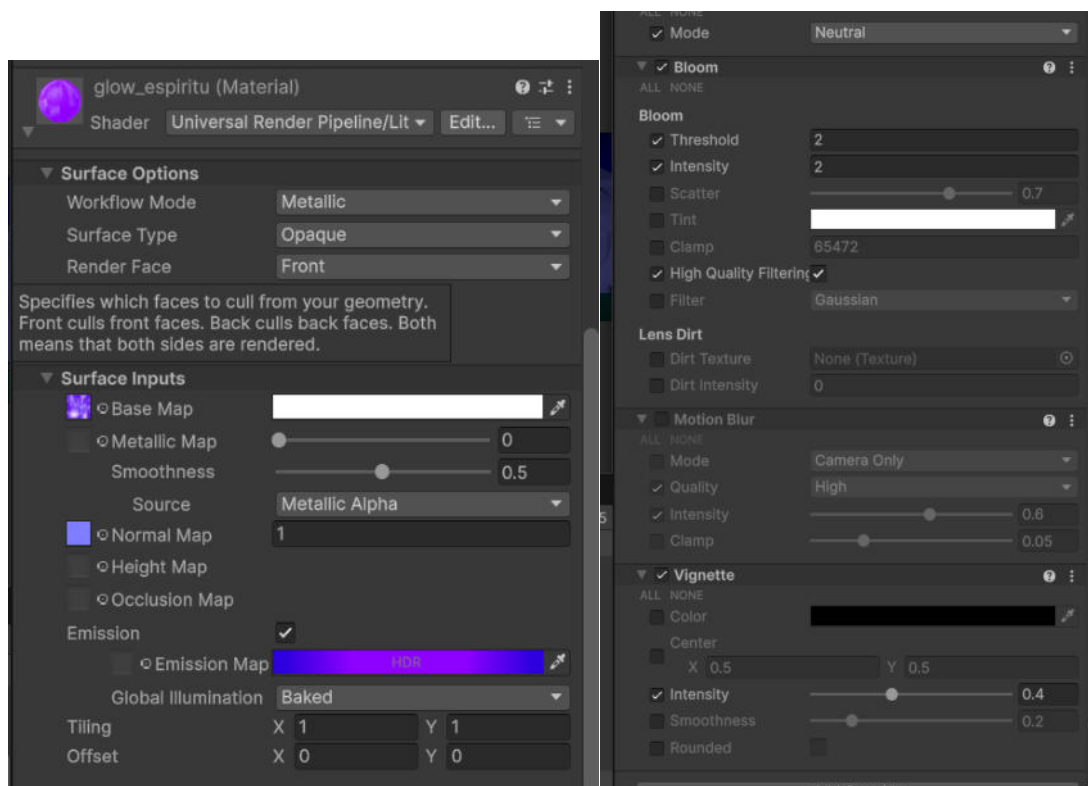
Utilice point lights para el fuego y el espíritu y los enemigos en su centro. Sin embargo, al estos estar en movimiento fueron colocados como mixed. De forma que podemos observar que se refleja la luz alrededor.



Postprocesado

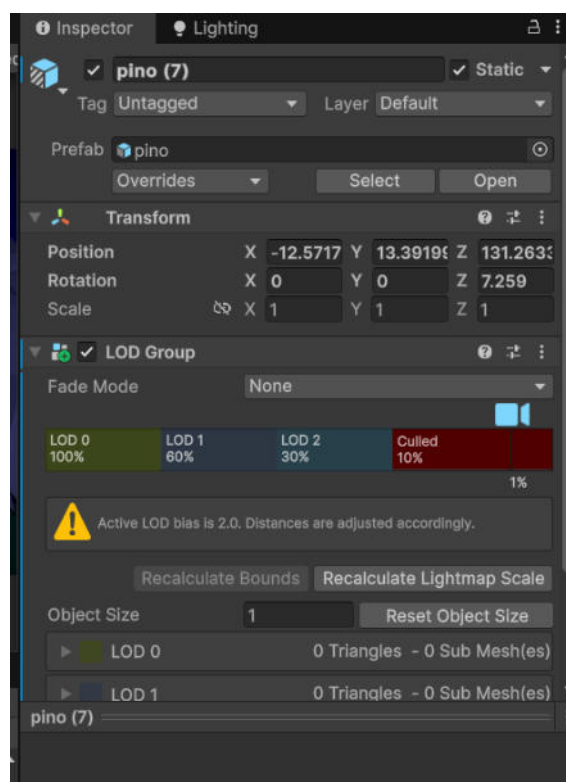
Para el cumplimiento del uso de posprocesamiento se hizo uso de Bloom, debido a que teníamos unos modelos que simulaban ser espíritus. Estos para darles un efecto más fantástico decidí que emitiera una luz y brillaran en la oscuridad. Para eso aplique lo que fue Bloom, además ayudaba con la intensidad de las luces que hacían las fogatas. Y eso también se uso en su material con ayuda del emisión Map.





Optimización y rendimiento

Varios de los elementos de plantas se utilizaron lo que es LOD.

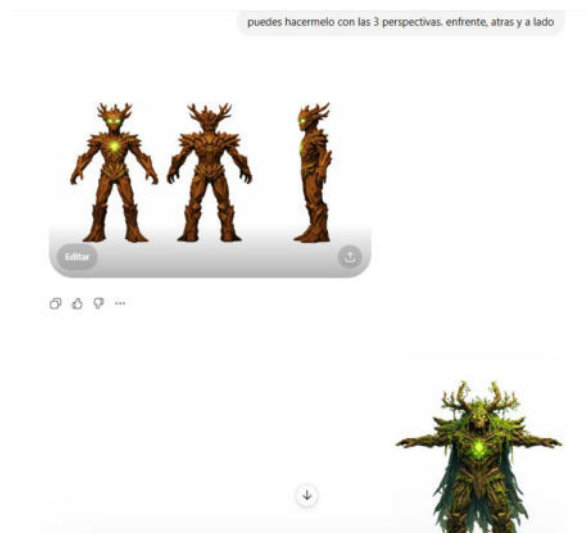


Arquitectura del proyecto

Modelos

Personajes

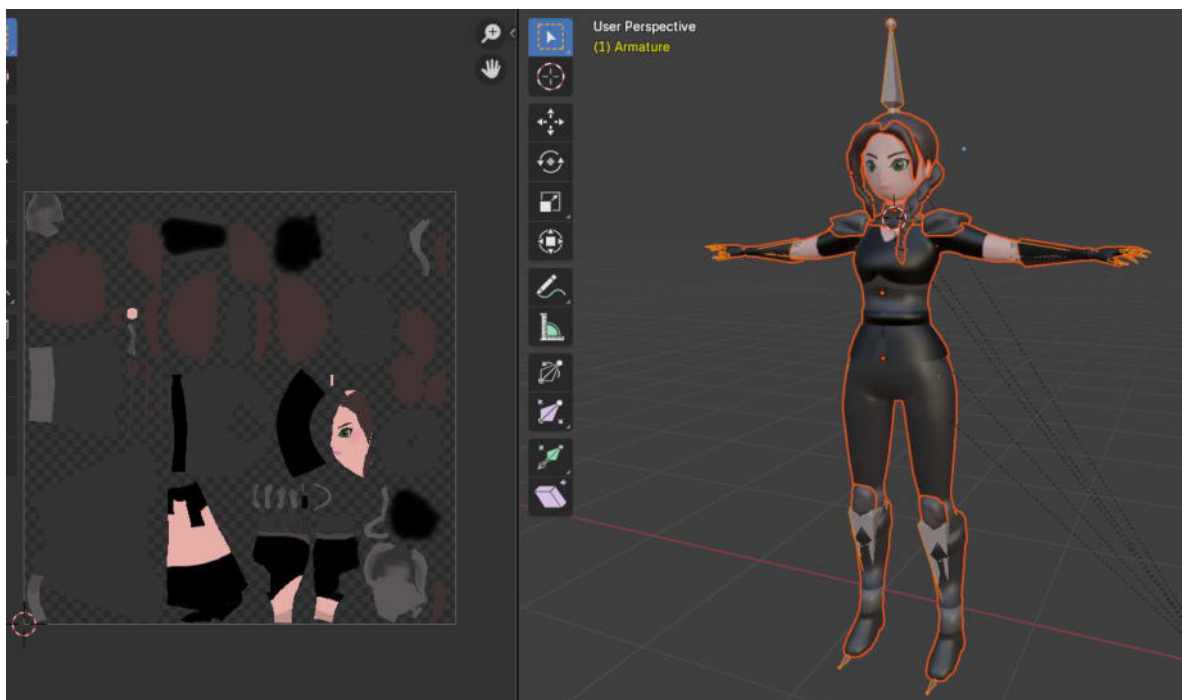
Todos los modelos fueron desarrollados inicialmente a partir de bocetos conceptuales realizados como referencia visual para definir la apariencia de los personajes, escenarios y elementos del entorno. Posteriormente, estos bocetos fueron procesados mediante herramientas de inteligencia artificial, las cuales permitieron transformar las ilustraciones base en modelos con apariencia tridimensional, facilitando así el proceso de diseño y visualización en 3D.



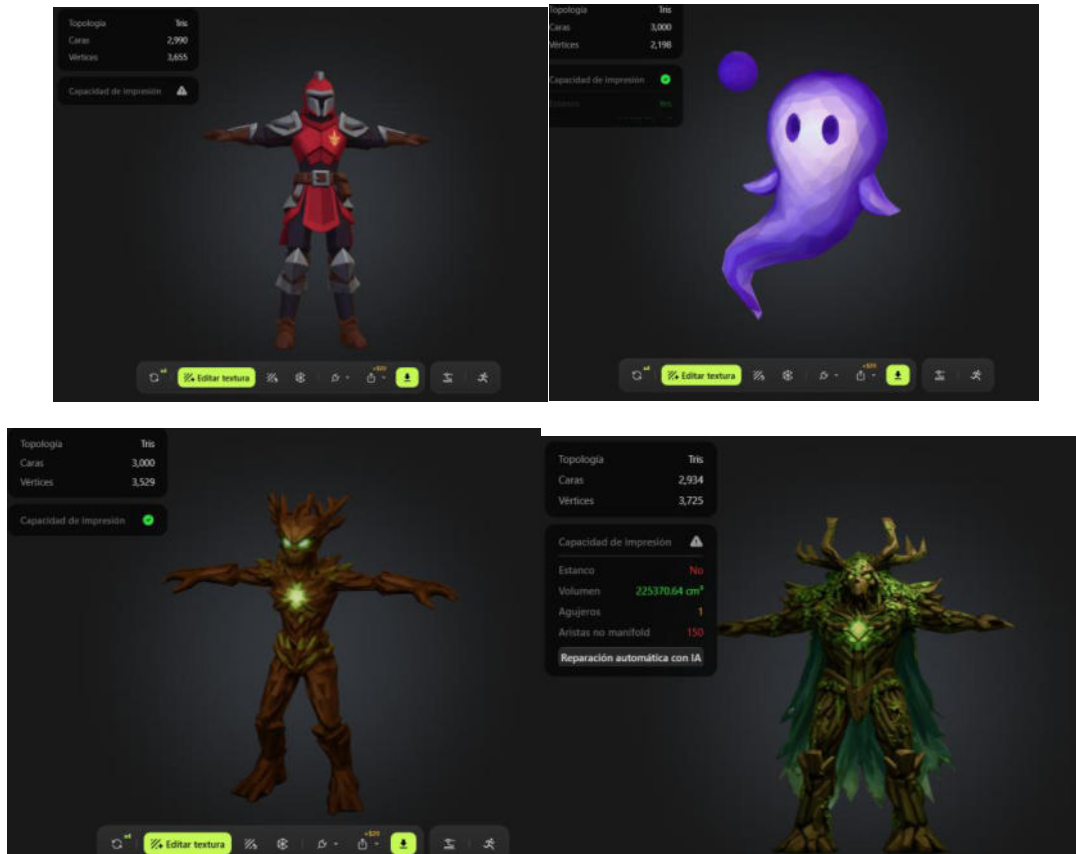
Además, el personaje principal o protagonista fue diseñado y creado completamente desde cero, desde su concepto inicial hasta la definición de sus características visuales, el modelado fue a través de Blender.



Al igual este modelo fue pintado en blender, guiándome con referencias e intentos para lo que fue el caso del rostro.



Los demás modelos como se mencionó anteriormente se mencionaron fueron hechos con Meshy.ia. Para generarlos se colocaron las imágenes generadas por CHATGPT y la página genera los modelos con la posibilidad de bajar sus caras y colocar su textura.

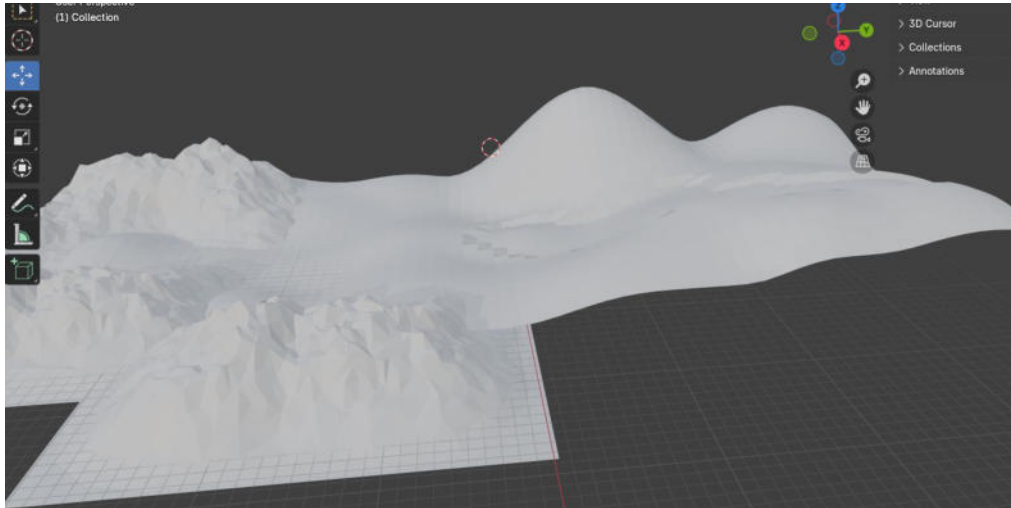


Algunos de estos modelos al ser hechos con Inteligencia Artificial no son completamente acertados, tuve en algunos que editarlos por ejemplo al soldado me hizo el cinturón en ambos lados y al espíritu le dibujo la cara en ambos lados. Por lo que solo cambié su geometría y moví el UV que tenía mapeado en su textura.

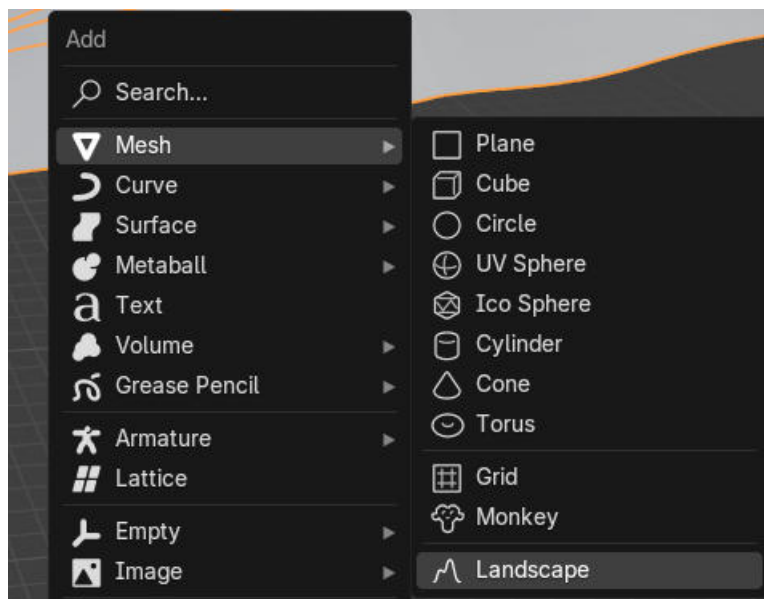


ESCENARIO

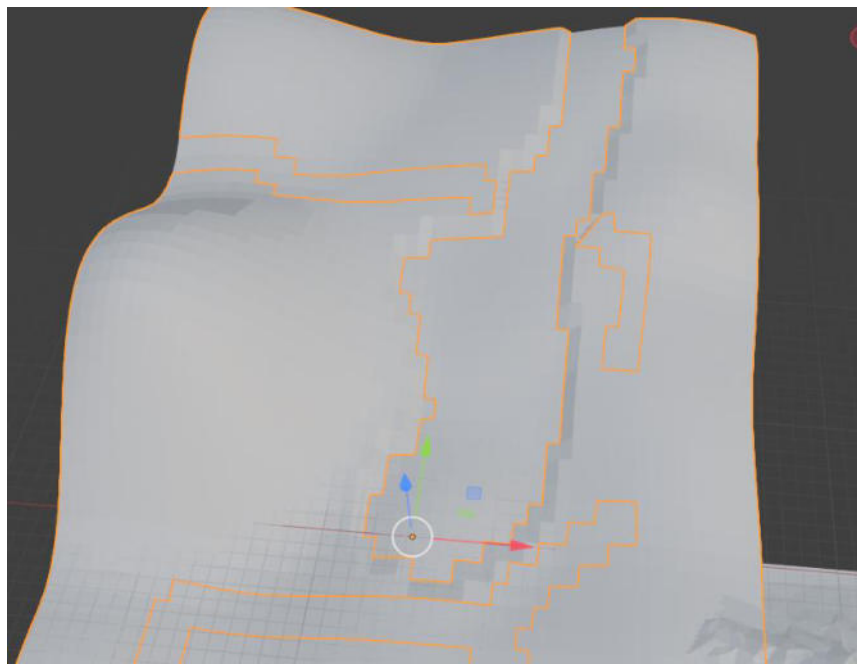
Para el escenario cree un plano en el cual lo subdividí en varias partes de forma que pudiera mover un vértice y con ayuda de la herramienta de Proportional Editing Objects se terminaba haciendo un tipo montaña por lo que fue sencillo el modificarlo para darle uniformidad al plano, ya que en el bosque no es completamente plano.



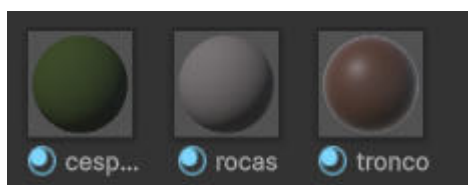
Y para darle más realismo use otra llamada landscapes y hacer unas montañas de rocas.



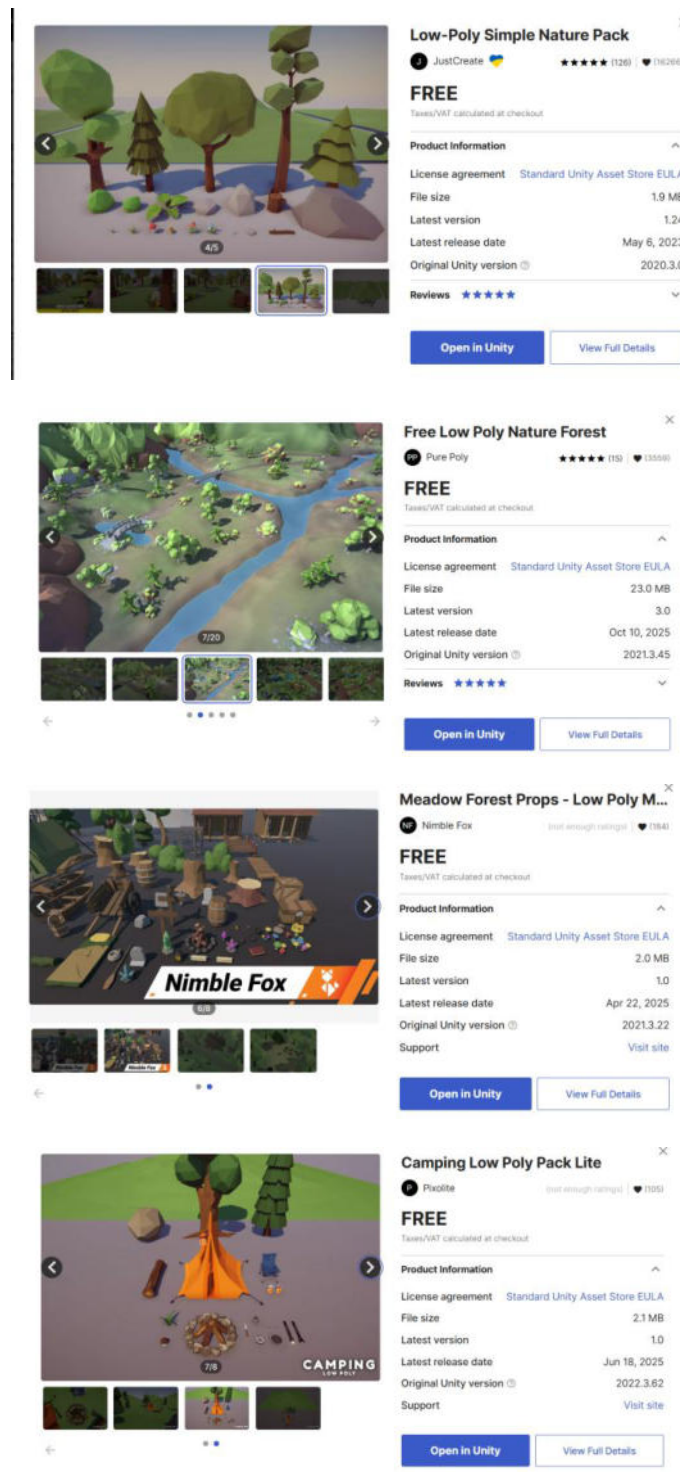
Para hacer el camino solo se seleccionó las caras y con Control+P se separaron del plano. Al igual que la parte donde sería el río.



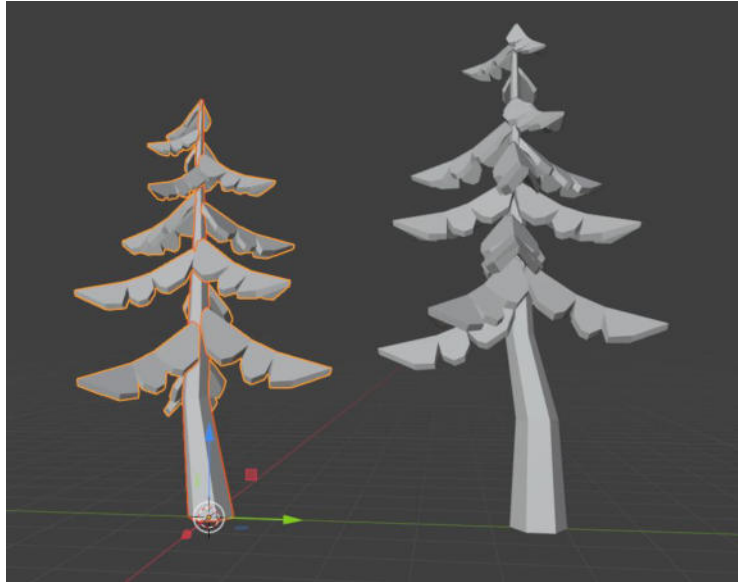
Para terminar con la base del escenario, al principio se utilizaron imágenes de texturas; sin embargo, con la integración de los modelos el resultado visual se veía extraño. Al observar ejemplos de bosques con estilo low poly, se identificó que estos utilizan colores planos, por lo que el modelo se pasó a Unity y se optó por crear ahí mismo los materiales para posteriormente asignárselos.



Los modelos de los escenarios la mayoría se consiguieron de los prefabs obtenidos en la tienda de Unity. Por lo que se muestran a continuación cuáles fueron usados:

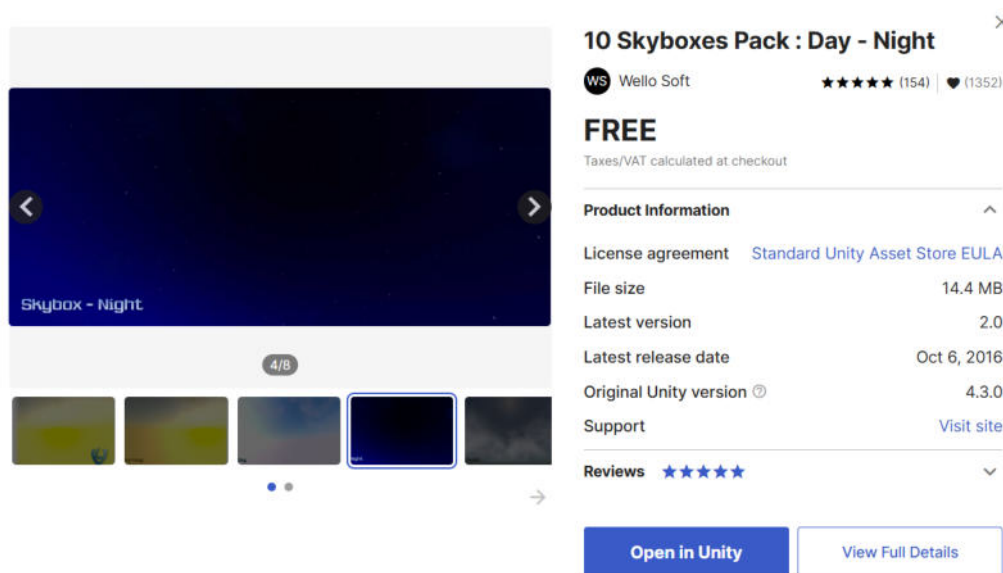


Varios de estos assets, como fue el caso de los árboles, requerían mantener el mismo tono de verde característico del bosque. Sin embargo, la mayoría de los modelos contaban únicamente con una sola malla, por lo que fue necesario separar las mallas en Blender para poder asignar los materiales deseados de manera independiente.

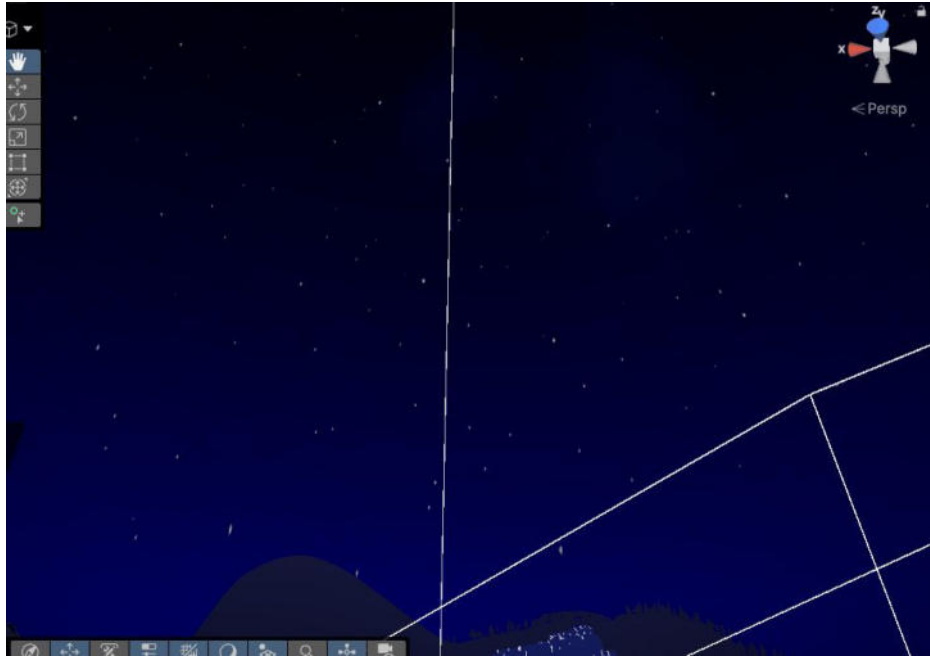


SKYBOX

Se buscó crear un ambiente nocturno que transmitiera tranquilidad y armonía con el entorno natural del proyecto. Para ello, fue necesario utilizar un skybox con temática nocturna que incluyera estrellas visibles, pero manteniendo una iluminación equilibrada para evitar que el escenario se percibiera demasiado oscuro. Por lo que también se descargó en la tienda de Unity:

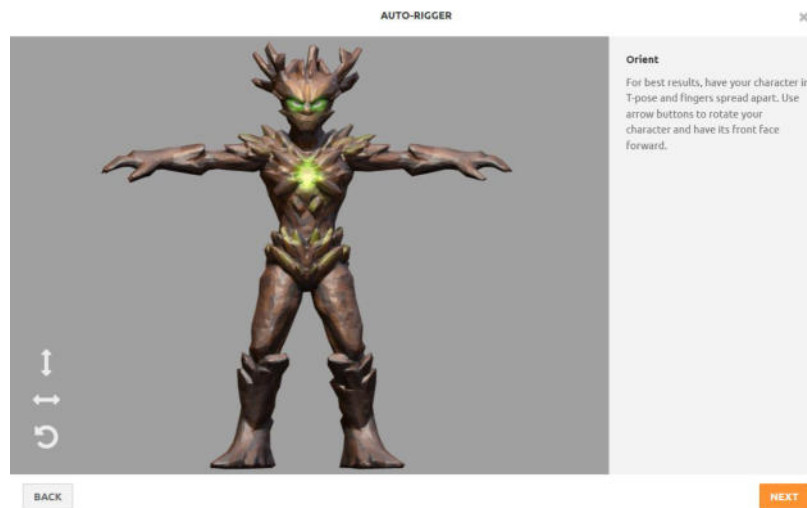


Y el resultado en la escena fue la siguiente:



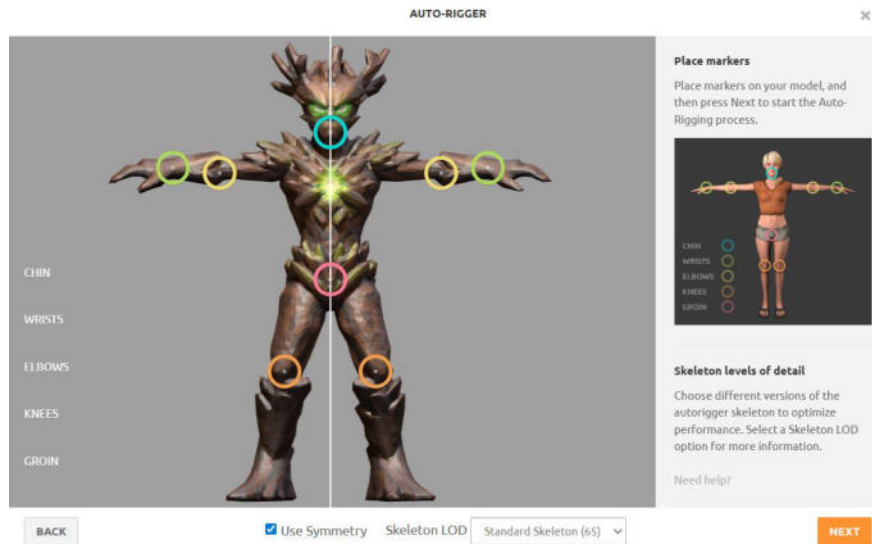
RIG

Para los modelos al tener la forma humanoide se utilizó la plataforma Mixamo, ya que permite realizar el rigging de manera automática y facilita el uso de las animaciones disponibles dentro de la misma herramienta.



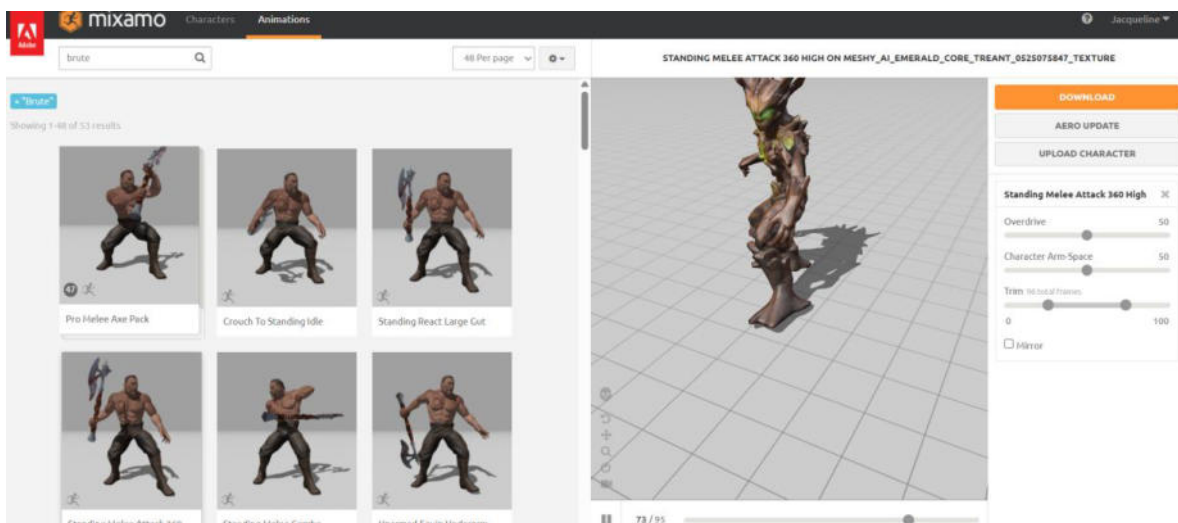
En Mixamo, para generar el esqueleto del modelo, es necesario asignar manualmente distintas partes del cuerpo que la plataforma solicita, como la barbilla, muñecas, codos, etc. Este proceso permite que el sistema identifique correctamente la

estructura del personaje y pueda crear automáticamente un rig compatible con las animaciones disponibles.



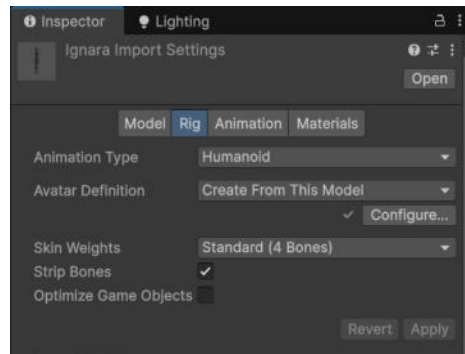
ANIMACIÓN

Para las animaciones se utilizaron tres diferentes sitios. El primero fue Mixamo, específicamente las animaciones del avatar llamado “Brute”, las cuales fueron utilizadas principalmente para los enemigos debido a que sus movimientos y estilo encajaban con el comportamiento que se buscaba representar.



EN UNITY

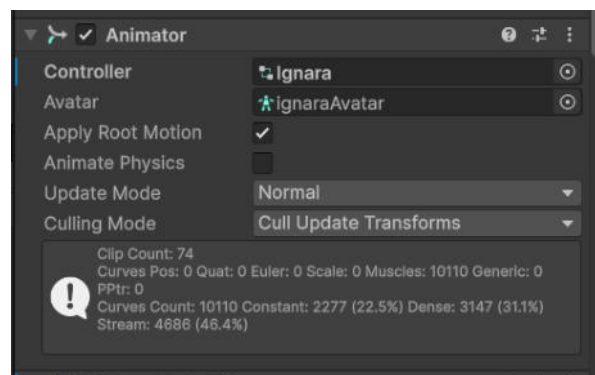
Para las animaciones en la configuración de cada modelo de la pestaña Rig, se selecciona el tipo de animación como Humanoid y la opción Create From This Model, lo que permite que Unity genere automáticamente el esqueleto o avatar del personaje. Esta configuración es necesaria para que el modelo pueda utilizar animaciones humanoides correctamente dentro del proyecto.



También es necesario el caso de los Animator Controller. Estos administran las animaciones de cada personaje, ya que permiten organizar y controlar los diferentes estados de animación, como caminar, atacar, correr o permanecer en reposo. Cada tipo de personaje contiene el propio, a excepción de los enemigos que comparten el mismo estilo de combate.

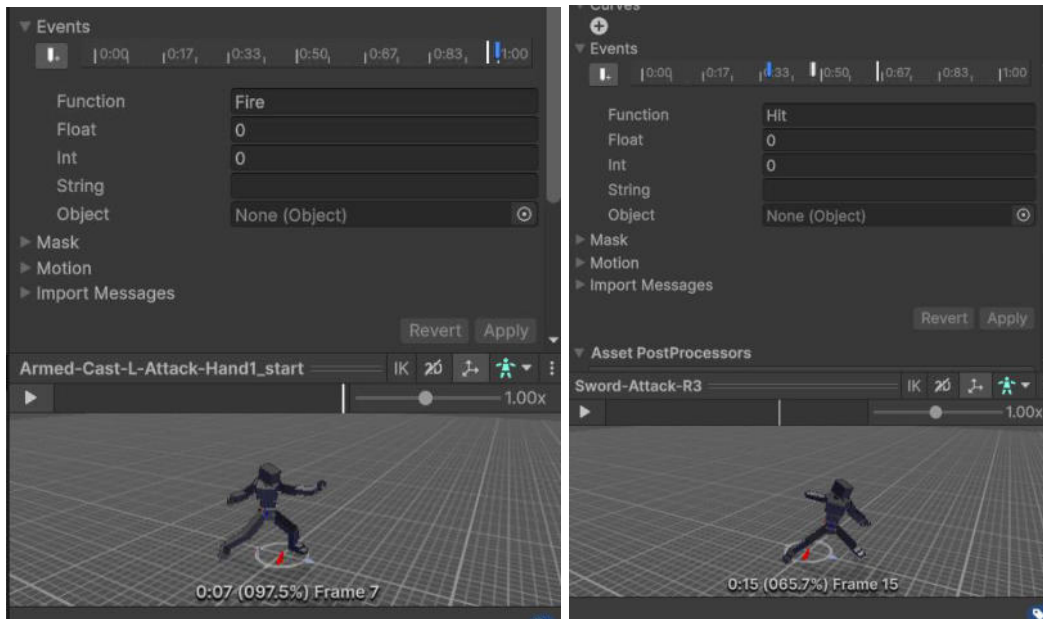


Por último, el componente Animator se asigna al modelo y es el encargado de reproducir las animaciones y enlazar el personaje con su respectivo Animator Controller y Avatar.

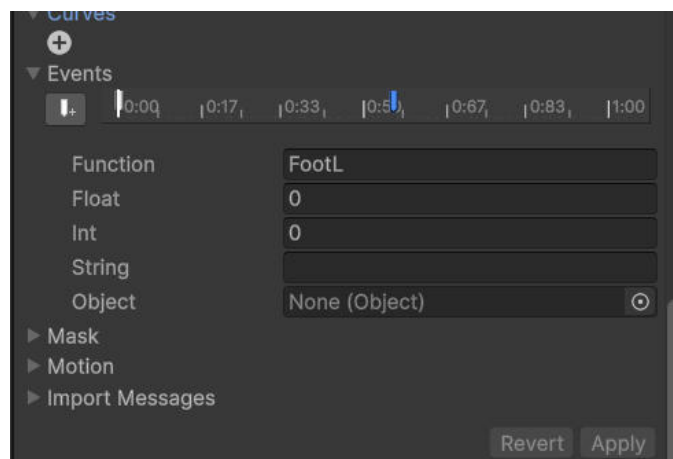


EVENTOS

Algunas animaciones, especialmente las de ataque o caminata, utilizan elementos llamados Events. Estos se colocan en una parte específica de la animación para activar acciones dentro del juego en el momento exacto. Por ejemplo, pueden usarse para lanzar un poder de fuego durante cierto punto de la animación, o para que el golpe de una espada ocurra justo cuando el movimiento del personaje lo indique, incluso para los efectos especiales.



Incluso para efectos como el sonido de las pisadas se pueden utilizar Events dentro de la animación. Estos eventos se colocan justo en los fotogramas donde los pies del personaje tocan el suelo, de manera que el sonido se reproduzca en el momento correcto. Esto ayuda a que el movimiento se sienta más natural y sincronizado con la acción del personaje.

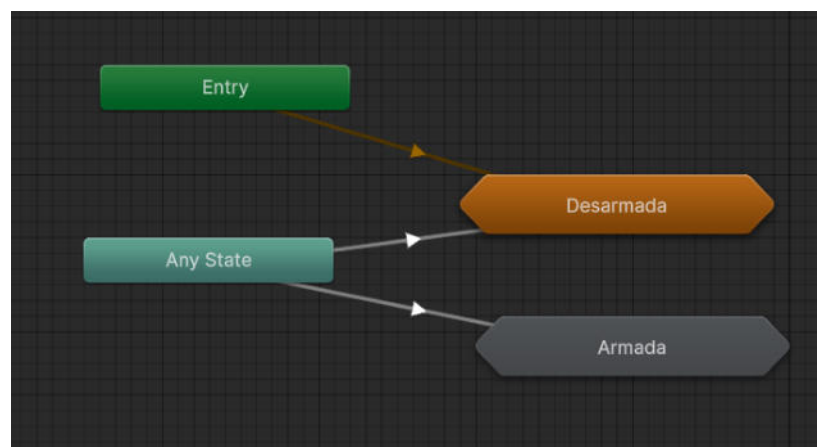


ARBOL DE ANIMACIÓN.

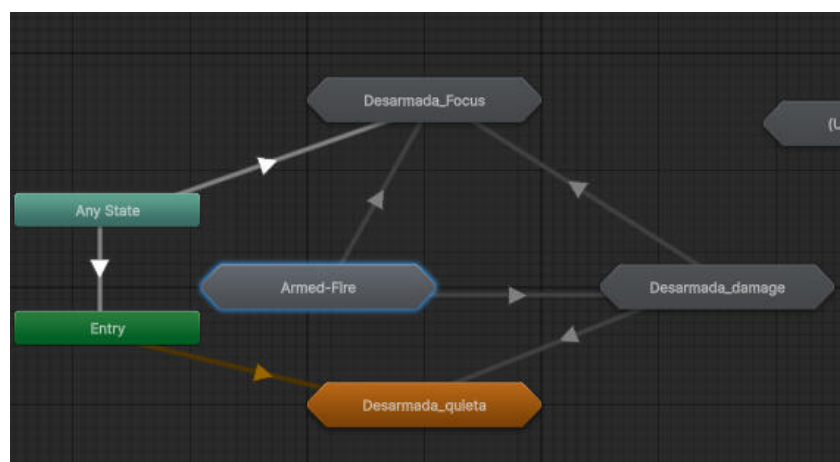
Los personajes cuentan con un árbol de animación, donde se organizan todas las animaciones que pueden realizar dependiendo de la acción que estén ejecutando. Este sistema permite cambiar entre movimientos como estar quieto, caminar, correr, atacar o recibir daño, según lo que ocurra dentro del juego.

Protagonista

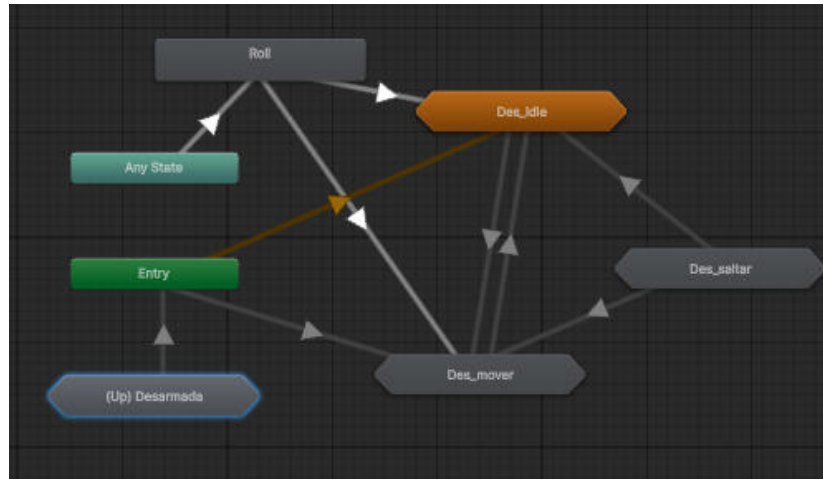
Para la protagonista se manejan dos subestados importantes dentro del árbol de animación: cuando está armada y cuando no lo está. Esto permite separar sus movimientos según la situación del personaje, ya que no realiza las mismas acciones al tener un arma equipada que al estar desarmada.



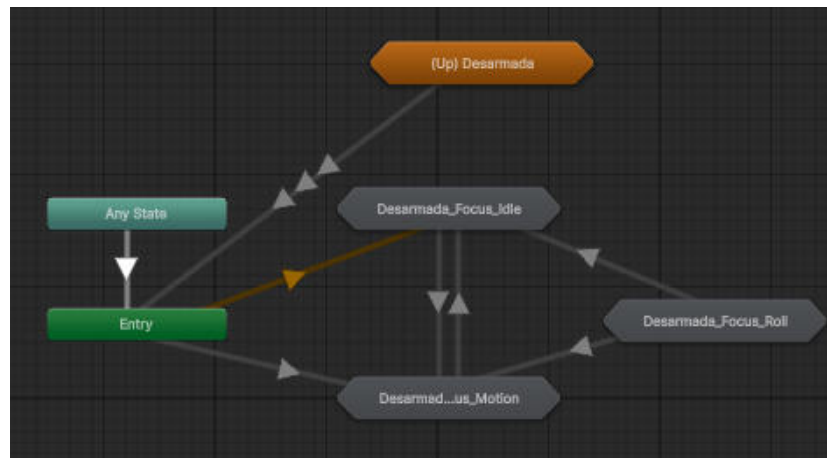
Posteriormente, cuando la protagonista se encuentra desarmada, el árbol de animación se divide en cuatro subestados principales: uno para el lanzamiento de fuego, otro para las animaciones al recibir daño, uno más para el modo de enfoque y finalmente el estado en el que el personaje permanece quieto o en reposo.



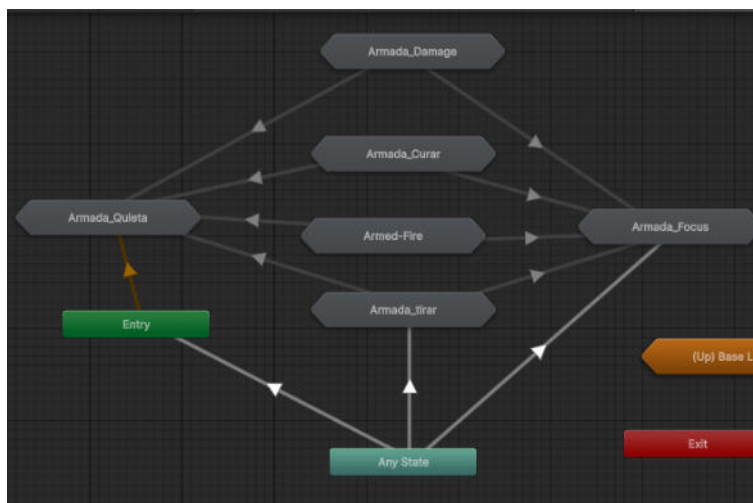
Las demás animaciones son más específicas, excepto dos casos principales, donde el estado de reposo incluye también acciones como esquivar y moverse. En un inicio se contempló la posibilidad de agregar la mecánica de salto; sin embargo, debido a que resultaba irrelevante para la jugabilidad y presentaba diversos errores, se decidió descartarla.



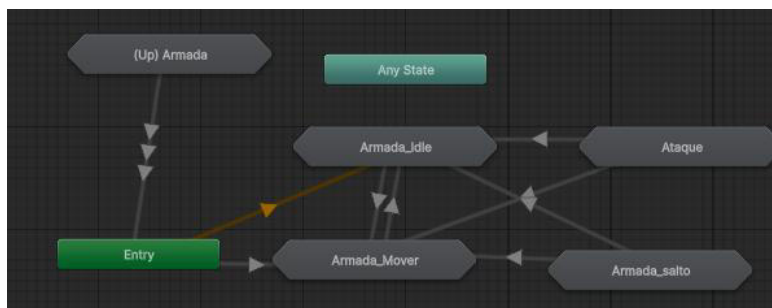
Y en el caso al estar el modo enfoque estas cambiaban a el Idle, esquivar y moverse.



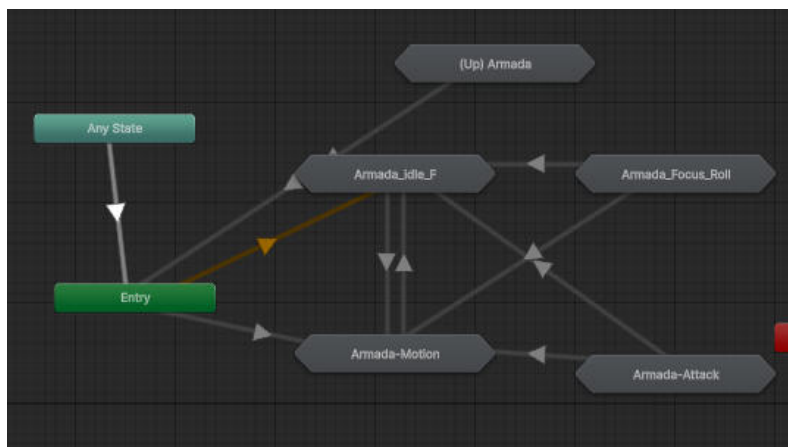
Para el caso de cuando se está armada es muy similar a cuando no. Tenemos casi la misma dinámica de animación, excepto que se agrega la curación.



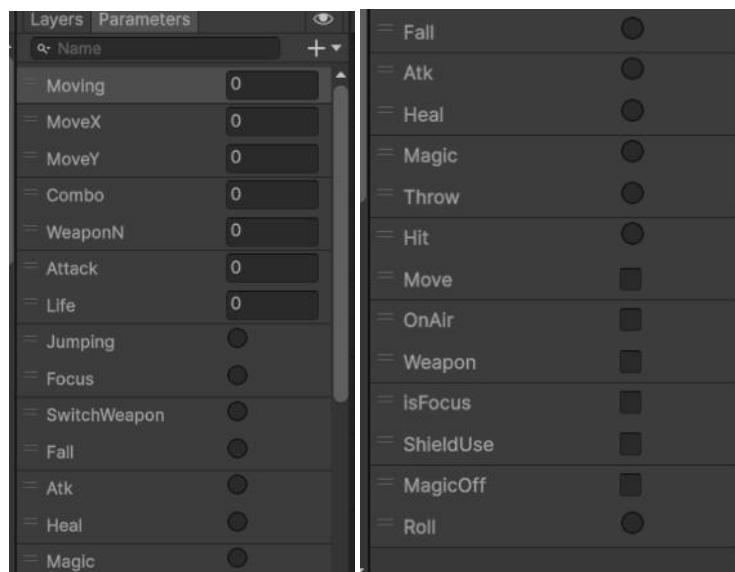
Y como se mencionó anteriormente, uno de los estados que aún contiene más mecánicas de animación no tan explícitas es el de “quieta”, ya que dentro de este se integran animaciones como el idle, el ataque, el movimiento y el salto, aunque esta última finalmente dejó de utilizarse.



Y para el estado focus sucede lo mismo con el mismo tipo de animaciones que se cuentan.

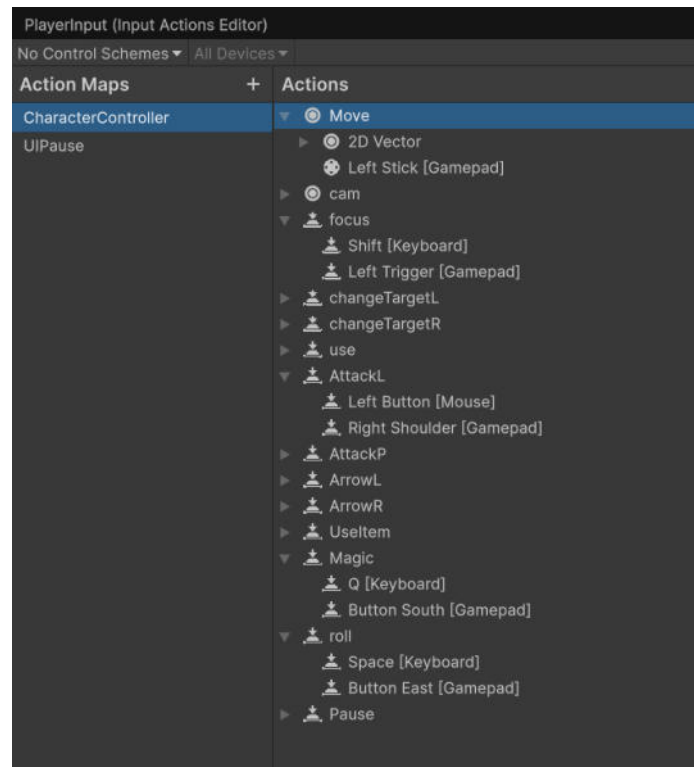


Y las transiciones funcionan mediante los parametros que se establecieron desde un inicio, por lo que algunos ya no se ocuparon. Pero se explica a continuacion que la idea general era la siguiente: Moving, MoveX y MoveY se relacionan con el movimiento; Attack, Atk y Combo con los ataques; Magic con el uso de habilidades como el lanzamiento de fuego; Hit con recibir daño; Roll con esquivar; y Weapon o WeaponN con el estado del personaje al estar armado o desarmado. Algunos parámetros, como Jumping, Fall u OnAir, fueron pensados para acciones relacionadas con el salto o la caída.



PLAYER INPUT

Se utilizó Player Input junto con el New Input System de Unity para poder mapear las acciones del personaje tanto en teclado como en control de juego. Esto permitió asignar diferentes entradas para una misma acción, como moverse, atacar, usar magia, rodar o pausar el juego, facilitando que el jugador pueda interactuar con el personaje desde distintos dispositivos.

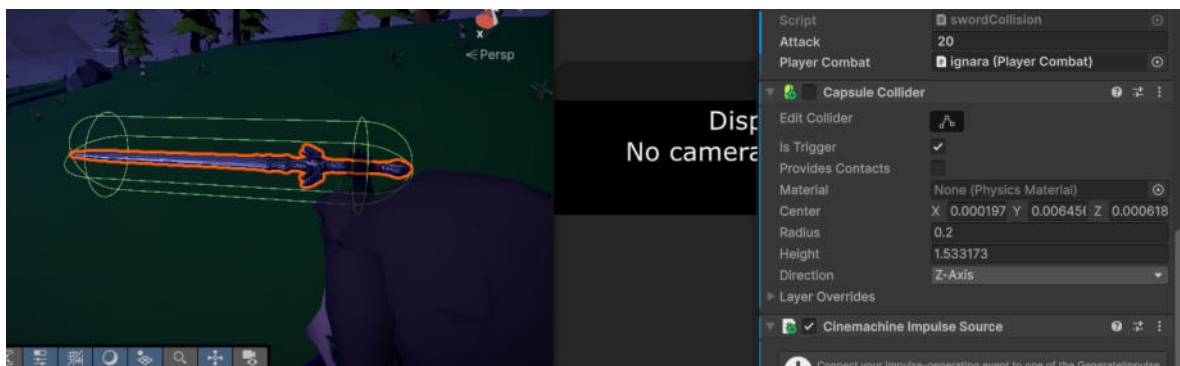


COMBATE

El combate en el personaje consta de dos tipos:

ESPADA

El combate con espada utiliza un modelo que necesita un collider y un script para detectar el contacto con los enemigos. Este script indica el daño que se ocasiona al enemigo y, como se mencionó en la sección de animaciones, depende de los Events para activar el golpe en el momento correcto de la animación. De esta forma, el daño no ocurre todo el tiempo, sino únicamente cuando el movimiento de la espada coincide con el ataque.

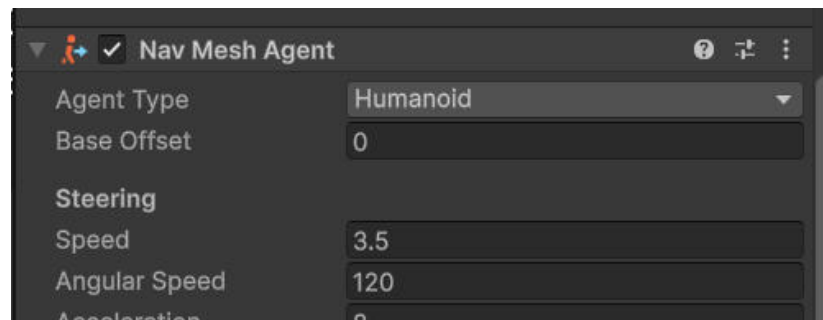


MAGIA

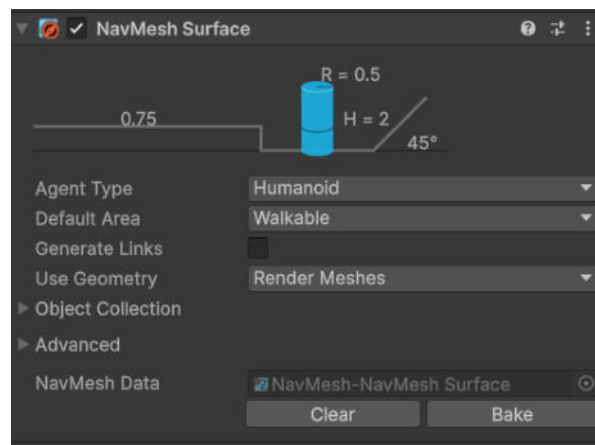
En el caso de la magia, se utiliza un prefab de fuego que se lanza desde el personaje. Este también cuenta con un sistema de colisión para detectar si impacta contra un enemigo o contra el suelo. Cuando el fuego toca a un objetivo, aplica daño y activa una explosión visual, además de generar un pequeño impulso en la cámara para dar mayor sensación de impacto. Esto permite que el ataque mágico se sienta más dinámico y tenga una respuesta visual clara dentro del juego.

ENEMIGOS

Para los enemigos se utilizó el componente Nav Mesh Agent, el cual permite que puedan moverse de forma automática dentro del escenario. Con este componente se configuran aspectos como la velocidad, la aceleración y el giro del enemigo, para que pueda desplazarse hacia el jugador o seguir una ruta dentro del mapa.

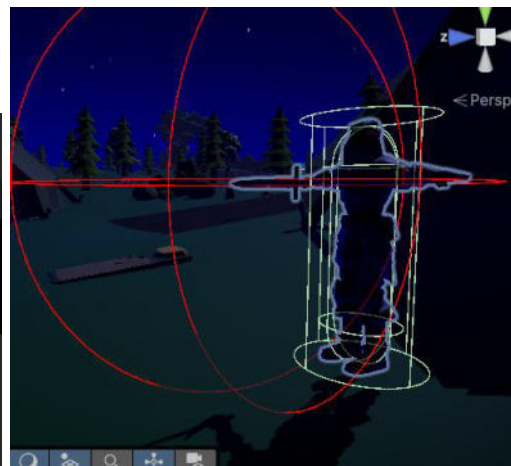


También se utilizó NavMesh Surface, que sirve para definir las zonas del escenario por donde los enemigos pueden caminar. Esta superficie se genera mediante el proceso de Bake, creando un área navegable que los enemigos reconocen para moverse correctamente sin atravesar objetos o salirse del terreno.



Para los enemigos también se implementaron scripts que controlan su comportamiento dentro del escenario. Estos permiten que el enemigo realice un patrullaje, moviéndose por determinadas zonas mientras no detecta al jugador. De esta forma, no permanece estático, sino que recorre el área para dar mayor dinamismo al juego.

Además, los enemigos cuentan con un sistema de visión, representado por un rango o campo de detección. Cuando el jugador entra dentro de esta zona, el enemigo puede identificarlo y comenzar a seguirlo o acercarse a él. Finalmente, el combate del enemigo se realiza mediante una espada, por lo que al estar cerca del jugador ejecuta su animación de ataque para causar daño.



INTERFAZ

La interfaz principal del juego fue diseñada para ser sencilla y fácil de utilizar. En ella se incluyen opciones como Continuar, Nueva Partida, Configuración y Salir.

La opción de Nueva Partida lleva directamente a la introducción del juego, permitiendo iniciar la historia desde el comienzo. Además, el botón de Continuar fue configurado para aparecer únicamente después de haber iniciado una partida previamente, dando la posibilidad de retomar el progreso guardado.

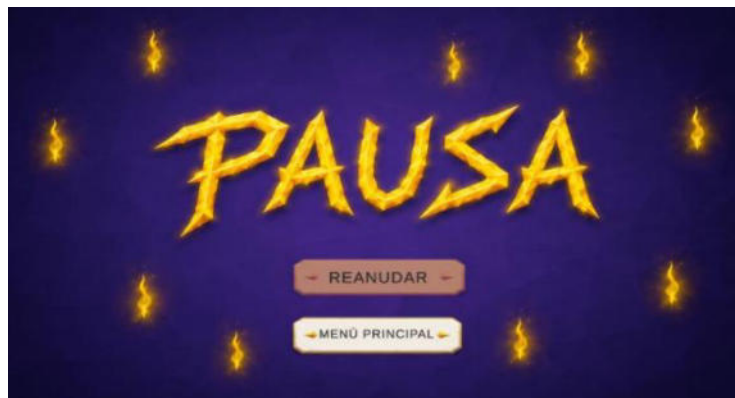
Por otro lado, el apartado de Configuración fue dejado preparado para futuras implementaciones, con la intención de añadir opciones relacionadas con sonido, controles y otros ajustes del juego.



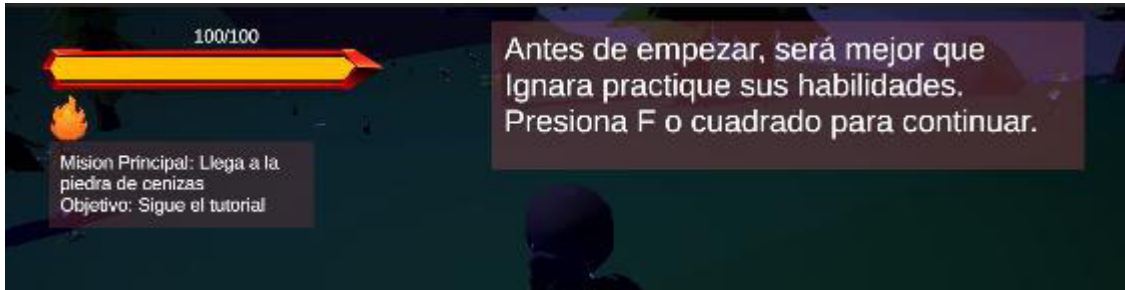
La interfaz de carga aparece durante el cambio entre escenas del juego, permitiendo que la transición se realice de forma más fluida mientras los elementos del escenario terminan de cargarse.



La interfaz de pausa puede activarse en cualquier momento durante la partida. Para esta sección también se utilizó el sistema de Player Input junto con el New Input System, permitiendo controlar la navegación de la interfaz tanto con teclado como con control de juego.

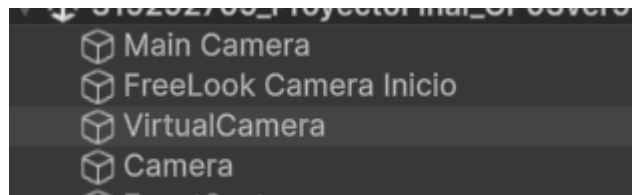


La interfaz principal durante la jugabilidad muestra distintos elementos importantes para el jugador, como la barra de vida, el tiempo de recarga de la magia de fuego representado mediante el icono de fuego, el apartado de misión principal y el tutorial inicial que guía al jugador durante los primeros momentos del juego.



cámaras

Dentro del proyecto se utilizaron varias cámaras para manejar diferentes funciones y perspectivas durante la jugabilidad. Entre ellas se encuentran la Main Camera, una FreeLook Camera Inicio y una VirtualCamera, las cuales permiten controlar distintos comportamientos de cámara dentro de la escena y facilitar las transiciones entre momentos específicos del juego.



La Main Camera es la cámara principal encargada de mostrar la escena al jugador.

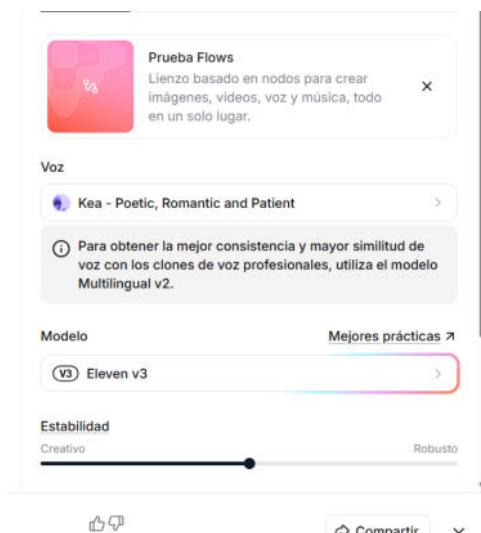
La FreeLook Camera Inicio se utiliza para el movimiento libre de la cámara alrededor del personaje durante la exploración y la jugabilidad normal.

La VirtualCamera se utiliza para momentos específicos dentro del juego, permitiendo cambiar la perspectiva o enfocar determinadas acciones según se requiera.

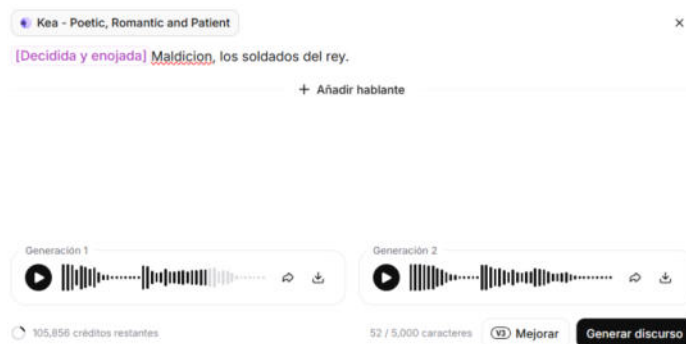
Finalmente, la cámara llamada Camera funciona como apoyo para controlar ciertas referencias de orientación y dirección dentro de la escena.

VOCES

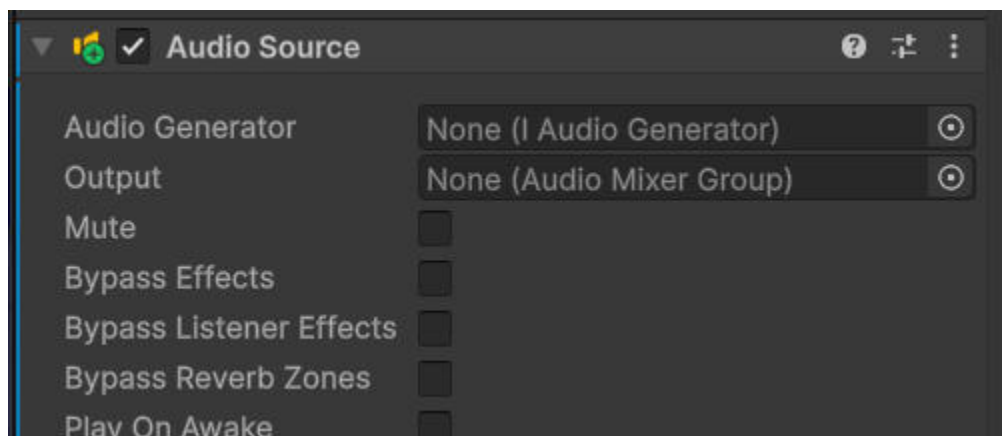
Para las voces de los personajes se utilizó la plataforma ElevenLabs, la cual permite generar voces mediante inteligencia artificial. Gracias a esta herramienta fue posible crear diálogos con diferentes tonos de voz, emociones y estilos de interpretación para los personajes del juego.



En este caso, la voz fue utilizada para la protagonista. Además, la herramienta permite modificar parámetros como estabilidad y expresividad para lograr una voz más natural y adecuada para cada diálogo.

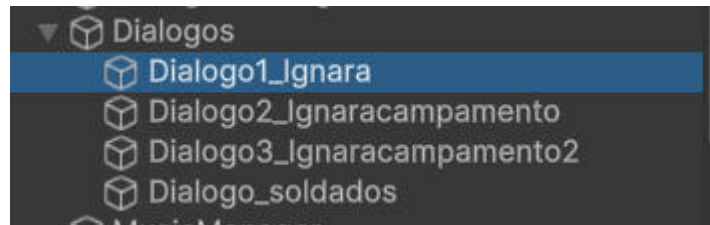


Para reproducir las voces dentro de Unity fue necesario utilizar el componente Audio Source. Este componente permite asignar los archivos de audio generados, como los diálogos de la protagonista, y reproducirlos dentro de la escena cuando sea necesario.

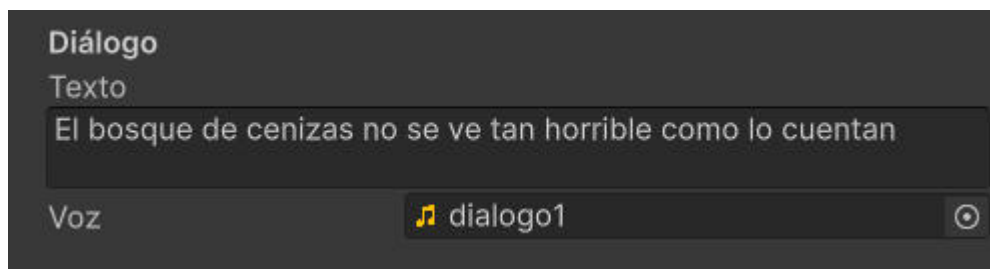


DIALOGOS

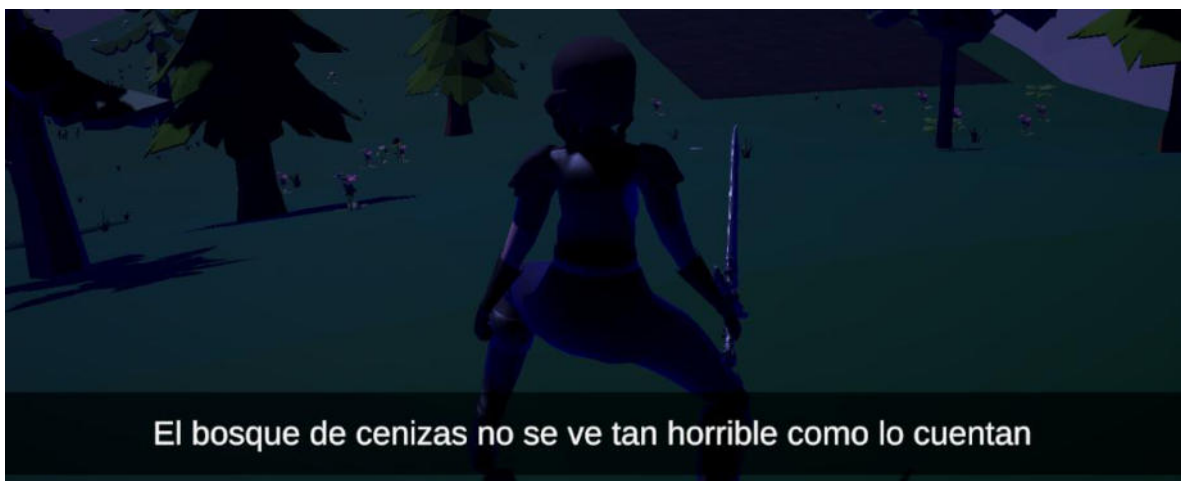
En ocasiones que el usuario llegue a áreas de dialogo se tiene la organización de los diferentes diálogos dentro de Unity. Cada diálogo se encuentra separado para poder identificar fácilmente en qué parte de la historia o escena será utilizado, facilitando así el control de conversaciones y secuencias narrativas del juego.



Cada diálogo se le asigna manualmente tanto el texto que aparecerá en pantalla como el archivo de voz correspondiente. Esto permite que la interfaz muestre el subtítulo al mismo tiempo que se reproduce el audio del personaje, ayudando a sincronizar correctamente los diálogos dentro de la escena.



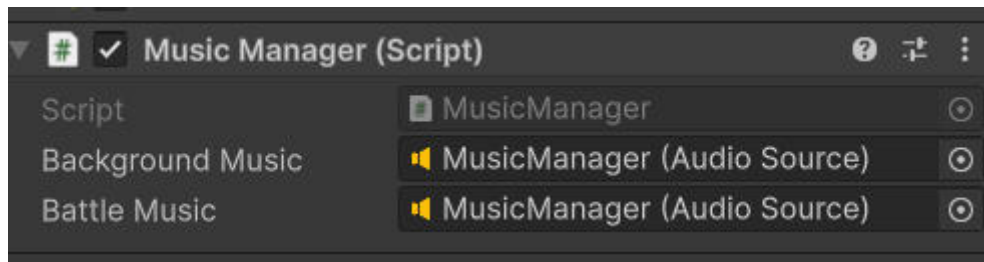
Los diálogos del juego se reflejan dentro de la interfaz mediante un cuadro de texto que aparece en pantalla junto con la voz del personaje.



Música

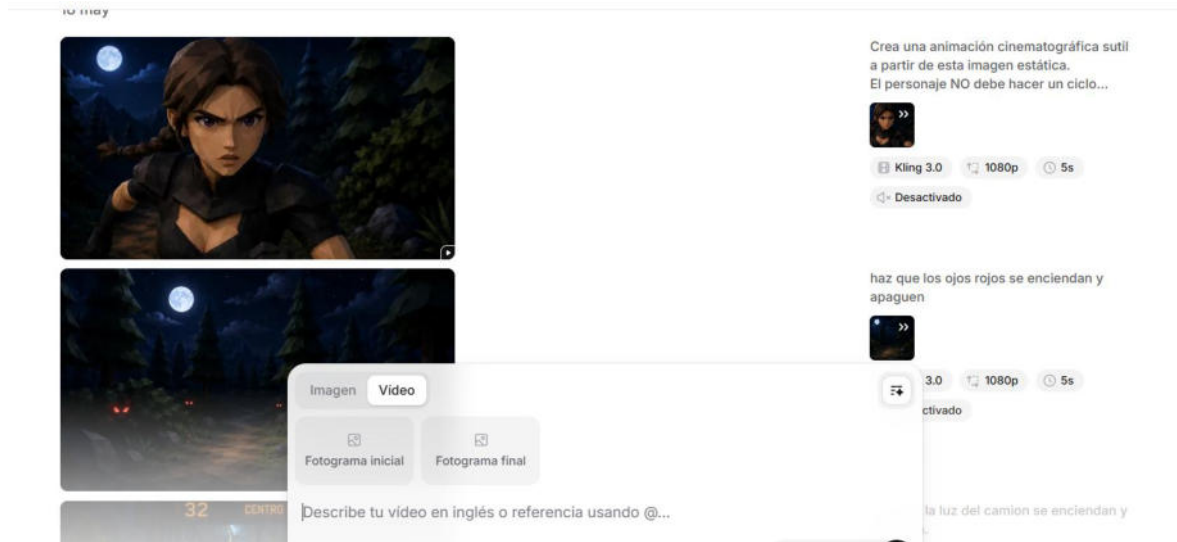
La música se controla con MusicManager, que funciona como un administrador central del audio del juego.

- Este script permite cambiar la música según lo que ocurre en la escena. Por ejemplo, cuando un enemigo detecta a la protagonista, se puede activar una música de combate o tensión. Cuando el enemigo pierde al jugador o muere, la música puede regresar a un estado normal o ambiental.
- También se conecta con otros scripts, como EnemyMotion y EnemyLife, que llaman funciones del MusicManager cuando el enemigo ve al jugador o cuando deja de perseguirlo.



VIDEO_INTRO

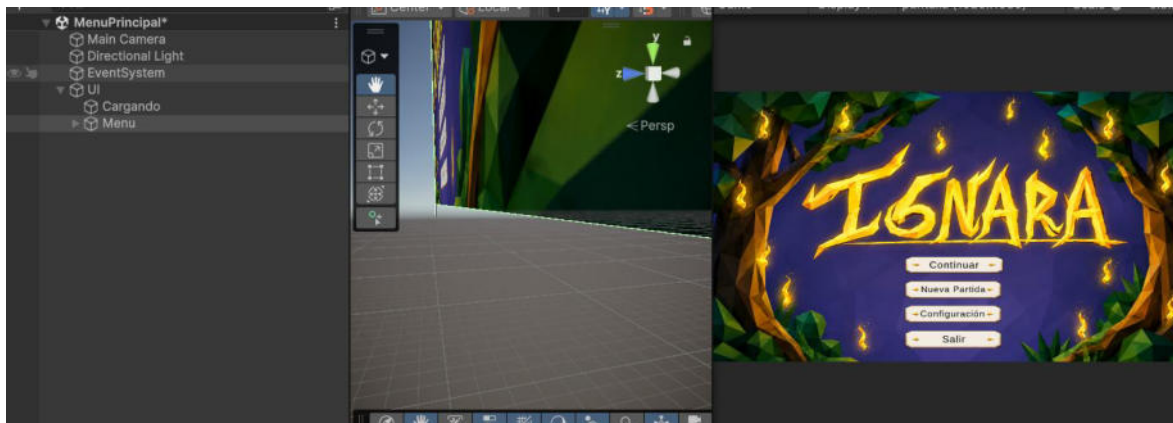
Además de las voces de los personajes, también se utilizó inteligencia artificial para apoyar en la creación de las animaciones de la introducción del juego. Mediante herramientas de generación visual basadas en IA, fue posible crear pequeñas secuencias cinemáticas a partir de imágenes estáticas, ayudando a darle mayor ambientación y estilo cinematográfico a la escena inicial.



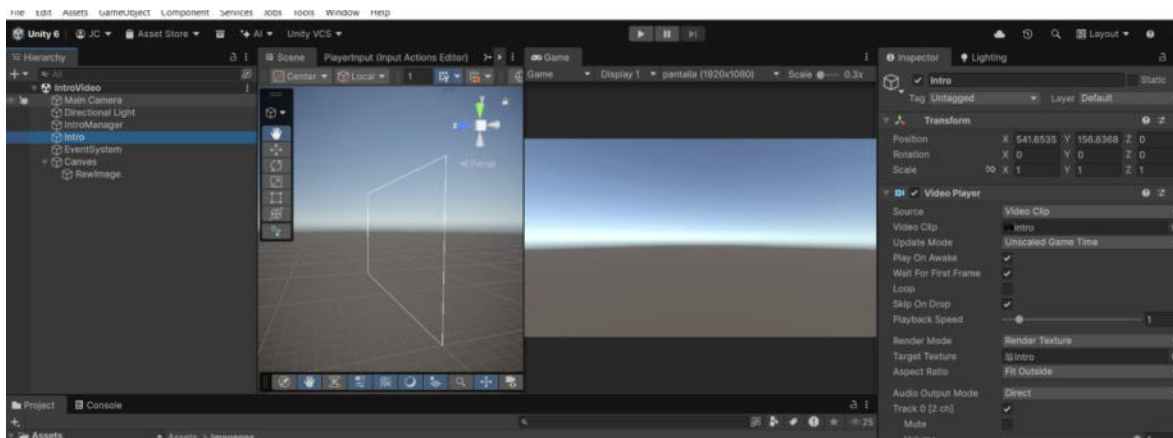
ESCENAS

Se tienen tres escenas en el proyecto:

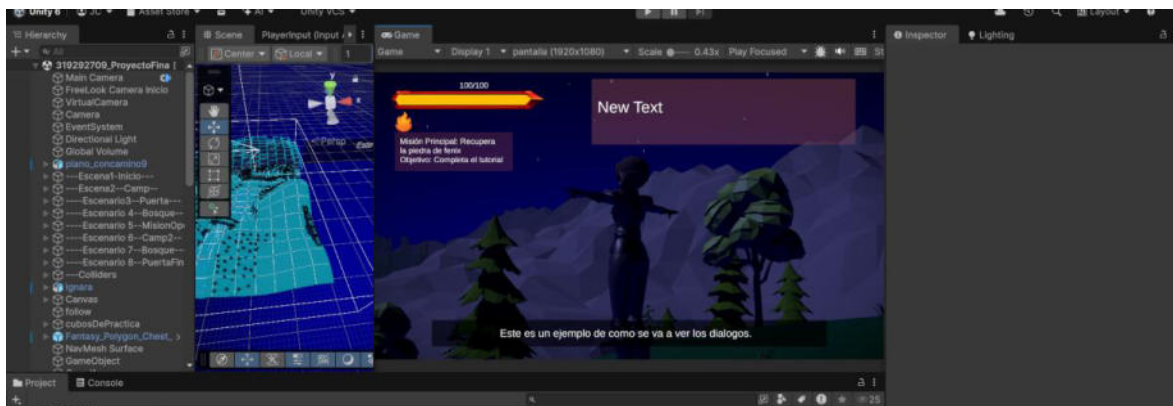
Escena para el menú:



Escena para la intro:



Escena del juego:



SCRIPTS

Enemigo

El funcionamiento del enemigo está dividido en varios scripts que trabajan juntos para controlar el movimiento, detección, combate y vida del enemigo.

- El script principal de comportamiento es EnemyMotion, el cual funciona como una máquina de estados. El enemigo puede estar patrullando, alerta, persiguiendo al jugador, atacando o buscando. Durante la patrulla recorre waypoints usando NavMeshAgent, y constantemente revisa si detecta al jugador mediante un OverlapSphere y un Raycast para verificar distancia, ángulo de visión y obstáculos. Cuando detecta al jugador cambia al estado de persecución y aumenta su velocidad de combate.
- Cuando el jugador entra en rango, el enemigo cambia al estado de ataque y ejecuta enemyCombat.Attack(). Mientras ataca, el enemigo deja de moverse, gira continuamente hacia el jugador y evita repetir ataques si ya está atacando o si el ataque fue bloqueado temporalmente.
- El script EnemyCombat controla los ataques cuerpo a cuerpo. Al iniciar el ataque activa animaciones mediante el Animator. Después, usando OverlapSphere, detecta si el jugador está dentro del área de impacto de la espada (swordPoint y swordArea) y aplica daño con PlayerLife.GetHit(). También incluye temporizadores para evitar ataques infinitos y un sistema que bloquea temporalmente los ataques cuando el enemigo recibe daño.
- EnemyCrossbowCombat hereda de EnemyCombat, pero reemplaza el ataque melee por un sistema de disparo. Instancia una flecha (arrowPrefab), la orienta hacia el jugador usando LookAt, le aplica fuerza con Rigidbody.AddForce y destruye la flecha después de unos segundos.
- El script EnemyEvent sirve como puente entre las animaciones y el código. Los eventos de animación llaman funciones como Hit(), Shoot() y EndAttack() para sincronizar exactamente el momento en que el enemigo golpea, dispara o termina una animación de ataque.
- Finalmente, EnemyLife controla la vida del enemigo. Cuando recibe daño, detiene temporalmente su movimiento, reproduce animaciones de golpe, partículas y textos flotantes de daño usando DOTween. Si la vida llega a cero, desactiva el movimiento y combate, reproduce partículas de muerte y destruye el enemigo. También notifica al sistema de música cuando el enemigo pierde o encuentra al jugador.

Protagonista

La protagonista se controla principalmente con PlayerMotion, PlayerCombat y PlayerLife.

- PlayerMotion maneja el movimiento general: recibe el input, mueve al personaje con Rigidbody, rota según la cámara, detecta si está en el suelo, permite rodar/esquivar, usar focus para fijar enemigos y cambiar de objetivo. También controla interacciones con cofres, papeles o NPCs, evitando que la protagonista se mueva o interactúe cuando está detenida, atacando o en otra acción.
- PlayerCombat controla las acciones ofensivas y de uso de objetos. Permite ataques ligeros y pesados con espada, disparo con ballesta, cambio de arma, uso de pociones y magia de fuego. Cuando ataca, activa animaciones, bloquea temporalmente el movimiento y luego vuelve a liberar el control con moveAgain(). También maneja combos, impulso hacia el enemigo cuando está en focus y el cooldown de la magia.
- swordCollision es el encargado de aplicar daño con la espada. Cuando el collider de la espada toca un objeto con tag Target, busca su componente Life y le aplica daño. Además, genera un impulso de cámara para dar sensación de impacto.
- PlayerLife administra la vida de la protagonista. Cuando recibe daño, actualiza la interfaz, reproduce animación de golpe, partículas, vibración de cámara y detiene momentáneamente el combate. Si la vida llega a cero, desactiva movimiento y combate, reproduce la muerte y reinicia la escena.
- Life funciona como clase base para vida, tanto del jugador como de enemigos. Guarda la vida máxima, la vida actual y tiene el método general GetHit() para restar daño.
- items define los objetos del juego como ScriptableObjects. Ahí se guarda el tipo de objeto o arma, por ejemplo espada, ballesta, curación, además de su imagen, daño, mensaje y descripción.

NPC y Diálogos

- DialogueManager es el controlador principal del diálogo. Activa el panel de diálogo, coloca el texto en pantalla y reproduce un audio si la línea tiene voz. Si hay audio, el diálogo se cierra cuando termina; si no hay audio, se cierra

después de unos segundos. También puede reproducir varias líneas seguidas usando un arreglo de LineaDialogo.

- DialogueTrigger sirve para diálogos automáticos por zona. Cuando la protagonista entra a un collider con tag Player, se muestra un texto y audio. Puede configurarse para activarse solo una vez. Además, si está conectado con una secuencia de enemigos, espera a que termine el diálogo y luego avisa a SecuenciaDialogoEnemigos.
- NPCDialogue controla los diálogos interactivos de los NPC. Cuando la protagonista se acerca, aparece un texto de interacción. Si el jugador presiona la tecla de interacción, se oculta ese texto y se reproduce la conversación por líneas mediante DialogueManager.
- NPCDialogueTrigger detecta cuándo la protagonista entra o sale del área del NPC. Al entrar, activa el texto de interacción y le asigna ese NPC al PlayerMotion, para que la protagonista pueda interactuar con él. Al salir, oculta el texto y elimina la referencia del NPC.
- EsconditeTrigger se usa para una secuencia especial. Detecta si la protagonista entra o sale de una zona de escondite y avisa a SecuenciaDialogoEnemigos, indicando si la jugadora se escondió o dejó de estar escondida.

MINIJUEGOS

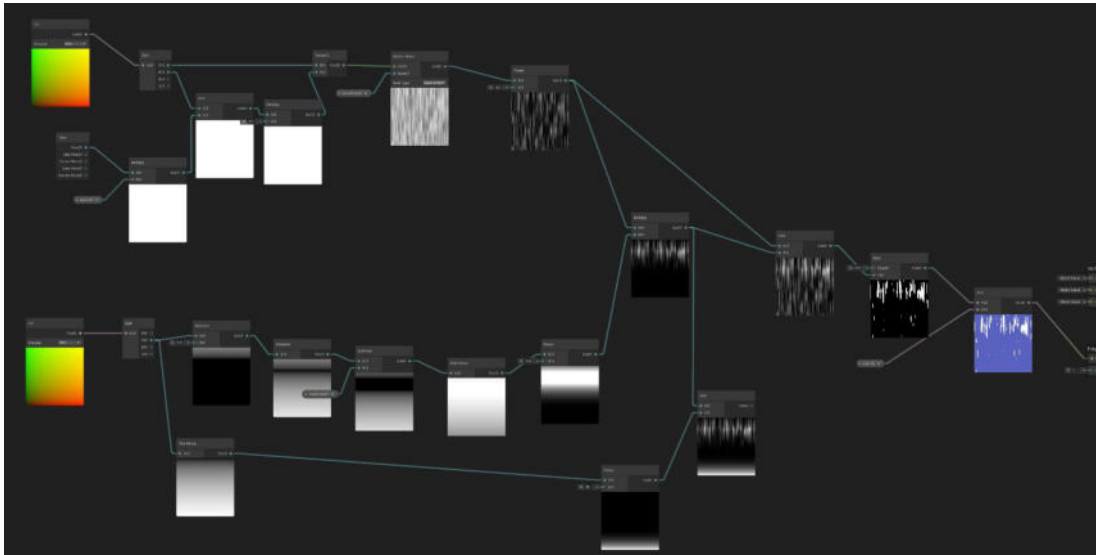
El minijuego consiste en un puzzle de memoria con cubos. Este se coloca como un reto para cruzar una puerta y continuar el juego. Primero se muestra un patrón iluminando los cubos en cierto orden, acompañado de sonidos. Después, el jugador debe repetir esa misma secuencia tocando los cubos correctos.

Si el jugador selecciona bien la secuencia, pasa a la siguiente ronda. Si se equivoca, los cubos cambian a color rojo y el patrón vuelve a mostrarse para intentarlo de nuevo. Al completar todas las rondas correctamente, la puerta se abre mediante un efecto de desvanecimiento, permitiendo avanzar en la misión.

SHADERS

Río

Como se mencionó al inicio se crearon varias versiones del agua, por lo que se tomaron varias técnicas para lograr un resultado de estilo caricatura. Este ya fue explicado anteriormente para lograr el efecto de agua y combinará en low poly.



Fuego

Este fue un elemento importado donde para el efecto de fuego se utilizó un shader enfocado en partículas, el cual permite darle una apariencia más dinámica y brillante. Este shader incluye configuraciones como distorsión, desvanecimiento y multiplicador HDR para intensificar el brillo del fuego dentro de escenas nocturnas.

Además, se utilizaron texturas y efectos de transparencia para que las partículas del fuego se vieran más suaves y con movimiento, ayudando a que la magia tuviera una apariencia más llamativa dentro del juego.



SISTEMA DE PARTICULAS

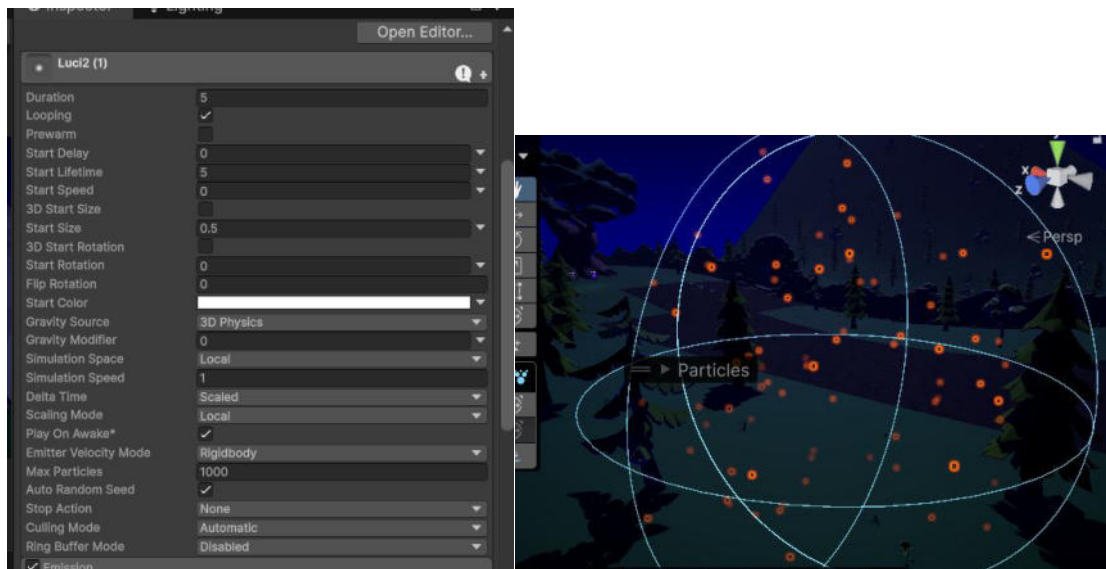
Luciérnaga

Para las luciérnagas también se utilizó un Particle System, buscando crear pequeños puntos de luz en movimiento dentro del bosque para reforzar el ambiente nocturno. En

este caso, las partículas se generan en un área amplia mediante una forma esférica, permitiendo que aparezcan distribuidas alrededor del escenario.

Además, se configuraron cambios de color, tamaño y ruido en el movimiento de las partículas para que las luciérnagas no se desplazaran de forma totalmente recta, dando una sensación más natural y orgánica.

Y el sistema incluye pequeñas luces mediante el apartado de Lights, permitiendo que cada partícula genere un ligero brillo alrededor, ayudando a ambientar mejor el bosque durante la noche.

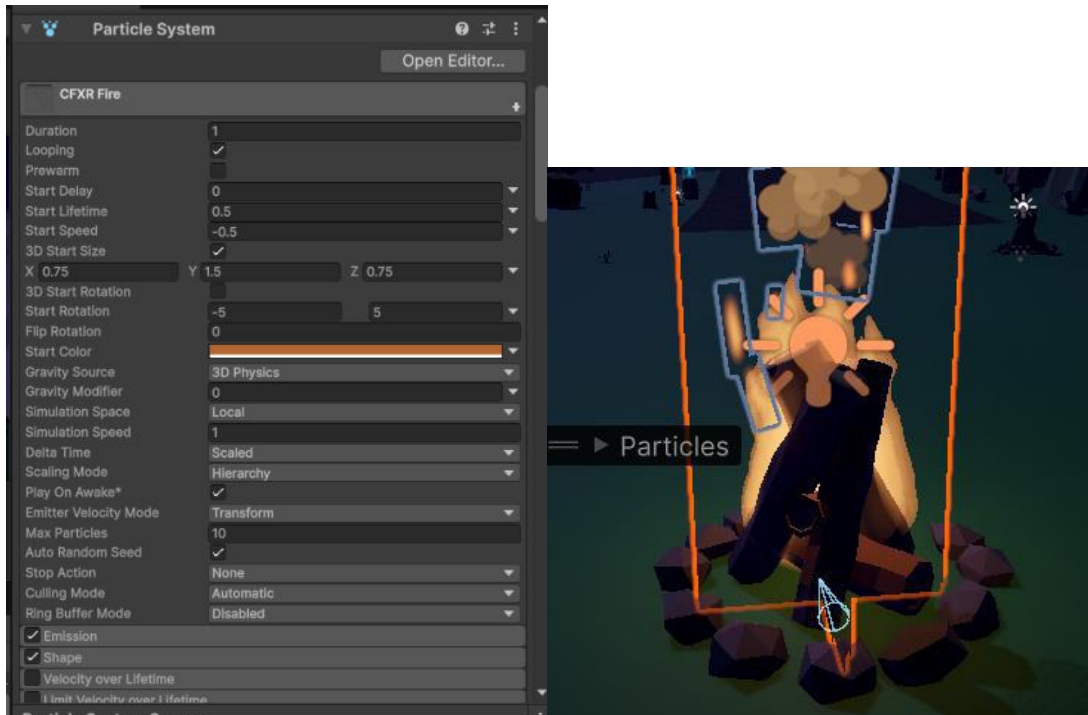


Fuego

Para los efectos de fuego se utilizó un Particle System compuesto por varios elementos que trabajan en conjunto para darle una apariencia más dinámica y realista. Dentro del sistema principal se configuraron aspectos como la duración de las partículas, velocidad, tamaño, emisión y dirección del fuego.

Además, el efecto no solo incluye las partículas principales de fuego, sino también sistemas adicionales como Ember para las chispas y Smoke para generar humo, ayudando a que el ataque mágico tuviera una apariencia más completa y visualmente atractiva.

También se añadió una Point Light, permitiendo que el fuego iluminara ligeramente el entorno y a los personajes cercanos. Esto ayuda a reforzar el efecto visual de la magia, especialmente dentro de los escenarios nocturnos del juego.



CONCLUSIONES

El desarrollo de este proyecto permitió aplicar de manera práctica diversos conocimientos relacionados con el desarrollo de videojuegos, integrando áreas como programación, modelado 3D, animación, shaders, inteligencia artificial, interfaces y efectos visuales dentro del motor Unity. A lo largo del proyecto se logró construir una demostración funcional de un videojuego estilo RPG con ambientación de fantasía, incorporando mecánicas de exploración, combate, interacción y narrativa.

Uno de los aspectos más importantes fue la integración de múltiples herramientas externas como Blender, Mixamo, ElevenLabs, Meshy y sistemas de partículas, lo que permitió complementar el flujo de trabajo y mejorar tanto la parte visual como sonora del proyecto. Asimismo, el uso de inteligencia artificial ayudó en la generación de modelos, voces y secuencias visuales, facilitando ciertos procesos creativos y técnicos.

Durante el desarrollo también se presentaron distintos retos, principalmente relacionados con el rendimiento, la optimización y la implementación de mecánicas más complejas. Sin embargo, mediante pruebas constantes y el enfoque de prueba y error, fue posible adaptar las mecánicas y mejorar progresivamente el funcionamiento del videojuego.